Computer Architecture
CS 622 / EE 520
Spring 2026

Lecture 15

Shahid Masud

Topics

- Advanced Topics in ILP, Multi-issue, and Speculation
- Some Cross Cutting Issues
- Architecture Study
  - ARM Cortex A53
  - Intel Core i7 6700

Midterm Next week

Assignment Due 9 Nov

Further Readings

Reading Notes

Readings:
Chapter 3 of course textbook

Exploiting ILP Using Dynamic Scheduling, Multiple Issue, and Speculation

ILP Using Dynamic Schedule, Multiple Issue and Speculation – Put all together

So far we have seen how the individual mechanisms of dynamic scheduling, multiple issue, and speculation work. In this section, we put all three together, which yields a microarchitecture quite similar to those in modern microprocessors. For simplicity we consider only an issue rate of two instructions per clock, but the concepts are no different from modern processors that issue three or more instructions per clock.

Let's assume we want to extend Tomasulo's algorithm to support multiple-issue superscalar pipeline with separate integer, load/store, and floating-point units (both FP multiply and FP add), each of which can initiate an operation on every clock. We do not want to issue instructions to the reservation stations out of order because this could lead to a violation of the program semantics. To gain the full advantage of dynamic scheduling, we will allow the pipeline to issue any combination of two instructions in a clock, using the scheduling hardware to actually assign operations to the integer and floating-point unit. Because the interaction of the integer and floating-point instructions is crucial, we also extend Tomasulo's scheme to deal with both the integer and floating-point functional units and registers, as well as incorporating speculative execution. As Figure 3.21 shows, the basic organization is similar to that of a processor with speculation with one issue per clock, except that the issue and completion logic must be enhanced to allow multiple instructions to be processed per clock.

Dealing with Multiple Instruction Issue per clock

Issuing multiple instructions per clock in a dynamically scheduled processor (with or without speculation) is very complex for the simple reason that the multiple instructions may depend on one another. Because of this, the tables must be updated for the instructions in parallel; otherwise, the tables will be incorrect or the dependence may be lost.

Two different approaches have been used to issue multiple instructions per clock in a dynamically scheduled processor, and both rely on the observation that the key is assigning a reservation station and updating the pipeline control tables.

1

One approach is to run this step in half a clock cycle so that two instructions can be processed in one clock cycle; this approach cannot be easily extended to handle four instructions per clock, unfortunately.

2

A second alternative is to build the logic necessary to handle two or more instructions at once, including any possible dependences between the instructions. Modern superscalar processors that issue four or more instructions per clock may include both approaches: They both pipeline and widen the issue logic. A key observation is that we cannot simply pipeline away the problem. By making instruction issues take multiple clocks because new instructions are issuing every clock cycle, we must be able to assign the reservation station and to update the pipeline tables so that a dependent instruction issuing on the next clock can use the updated information.

Multiple Issue Processor with Speculation

Superscalar

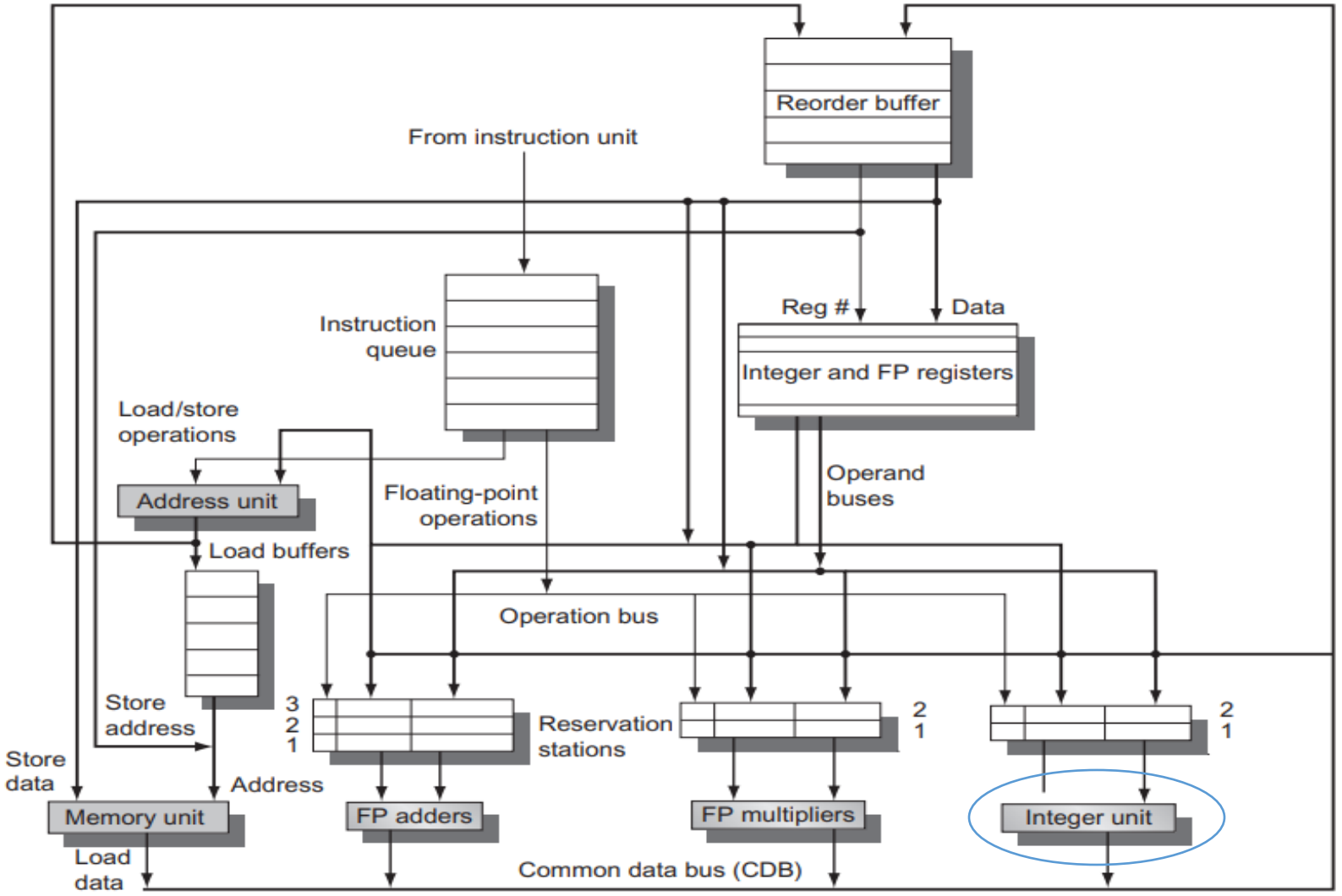


Figure 3.21 The basic organization of a multiple issue processor with speculation. In this case, the organization could allow a FP multiply, FP add, integer, and load/store to all issues simultaneously (assuming one issue per clock per functional unit). Note that several datapaths must be widened to support multiple issues: the CDB, the operand buses, and, critically, the instruction issue logic, which is not shown in this figure. The last is a difficult problem, as we discuss in the text.

Strategy for Dynamically Scheduled Superscalar

to describe the basic strategy for updating the issue logic and the reservation tables in a dynamically scheduled superscalar with up to *n* issues per clock as follows:

1. Assign a reservation station and a reorder buffer for *every* instruction that *might* be issued in the next issue bundle. This assignment can be done before the instruction types are known simply by preallocating the reorder buffer entries sequentially to the instructions in the packet using *n* available reorder buffer entries and by ensuring that enough reservation stations are available to issue the whole bundle, independent of what it contains. By limiting the number of instructions of a given class (say, one FP, one integer, one load, one store), the necessary reservation stations can be preallocated. Should sufficient reservation stations not be available (such as when the next few instructions in the program are all of one instruction type), the bundle is broken, and only a subset of the instructions, in the original program order, is issued. The remainder of the instructions in the bundle can be placed in the next bundle for potential issue.
2. Analyze all the dependences among the instructions in the issue bundle.
3. If an instruction in the bundle depends on an earlier instruction in the bundle, use the assigned reorder buffer number to update the reservation table for the dependent instruction. Otherwise, use the existing reservation table and reorder buffer information to update the reservation table entries for the issuing instruction.

Of course, what makes the preceding very complicated is that it is all done in parallel in a single clock cycle!

- ISSUE Step is a major bottleneck
- In a modern superscalar, every possible combination of dependent instructions that is allowed to issue in the same clock cycle must be considered.
- Because the number of possibilities climbs as the square of the number of instructions that can be issued in a clock, the issue step is a likely bottleneck for attempts to go beyond four instructions per clock.

Commit Strategy - continued

Of course, what makes the preceding very complicated is that it is all done in parallel in a single clock cycle!

At the back-end of the pipeline, we must be able to complete and commit multiple instructions per clock. These steps are somewhat easier than the issue problems because multiple instructions that can actually commit in the same clock cycle must have already dealt with and resolved any dependences. As we will see, designers have figured out how to handle this complexity: The Intel i7, which we examine in Section 3.12, uses essentially the scheme we have described for speculative multiple issue, including a large number of reservation stations, a reorder buffer, and a load and store buffer that is also used to handle nonblocking cache misses.

From a performance viewpoint, we can show how the concepts fit together with an example.

Observations from the Example (following slides)

Discussed separately in solved examples

Figures 3.23 and 3.24 show the performance for a two-issue, dynamically scheduled processor, without and with speculation. In this case, where a branch can be a critical performance limiter, speculation helps significantly. The third branch in the speculative processor executes in clock cycle 13, whereas it executes in clock cycle 19 on the nonspeculative pipeline. Because the completion rate on the nonspeculative pipeline is falling behind the issue rate rapidly, the nonspeculative pipeline will stall when a few more iterations are issued. The performance of the nonspeculative processor could be improved by allowing load instructions to complete effective address calculation before a branch is decided, but unless speculative memory accesses are allowed, this improvement will gain only 1 clock per iteration.

This example clearly shows how speculation can be advantageous when there are data-dependent branches, which otherwise would limit performance. This advantage depends, however, on accurate branch prediction. Incorrect speculation does not improve performance; in fact, it typically harms performance and, as we shall see, dramatically lowers energy efficiency.

Example page 226

Example Consider the execution of the following loop, which increments each element of an integer array, on a two-issue processor, once without speculation and once with speculation:

```
Loop:  ld    x2,0(x1)      //x2=array element
       addi  x2,x2,1       //increment x2
       sd    x2,0(x1)      //store result
       addi  x1,x1,8       //increment pointer
       bne   x2,x3,Loop    //branch if not last
```

Assume that there are separate integer functional units for effective address calculation, for ALU operations, and for branch condition evaluation. Create a table for the first three iterations of this loop for both processors. Assume that up to two instructions of any type can commit per clock.

Solution: 2 Issue Processor Without Speculation

Dynamically Scheduled

Iteration number	Instructions	Issues at clock cycle number	Executes at clock cycle number	Memory access at clock cycle number	Write CDB at clock cycle number	Comment
1	ld x2,0(x1)	1	2	3	4	First issue
1	addi x2,x2,1	1	5		6	Wait for ld
1	sd x2,0(x1)	2	3	7		Wait for addi
1	addi x1,x1,8	2	3		4	Execute directly
1	bne x2,x3,Loop	3	7			Wait for addi
2	ld x2,0(x1)	4	8	9	10	Wait for bne
2	addi x2,x2,1	4	11		12	Wait for ld
2	sd x2,0(x1)	5	9	13		Wait for addi
2	addi x1,x1,8	5	8		9	Wait for bne
2	bne x2,x3,Loop	6	13			Wait for addi
3	ld x2,0(x1)	7	14	15	16	Wait for bne
3	addi x2,x2,1	7	17		18	Wait for ld
3	sd x2,0(x1)	8	15	19		Wait for addi
3	addi x1,x1,8	8	14		15	Wait for bne
3	bne x2,x3,Loop	9	19			Wait for addi

Figure 3.23 The time of issue, execution, and writing result for a dual-issue version of our pipeline *without* speculation. Note that the ld following the bne cannot start execution earlier because it must wait until the branch outcome is determined. This type of program, with data-dependent branches that cannot be resolved earlier, shows the strength of speculation. Separate functional units for address calculation, ALU operations, and branch-condition evaluation allow multiple instructions to execute in the same cycle. Figure 3.24 shows this example with speculation.

Solution: 2 Issue Proc with Speculation

Dynamically Scheduled

Iteration number	Instructions	Issues at clock number	Executes at clock number	Read access at clock number	Write CDB at clock number	Commits at clock number	Comment
1	ld x2,0(x1)	1	2	3	4	5	First issue
1	addi x2,x2,1	1	5		6	7	Wait for ld
1	sd x2,0(x1)	2	3			7	Wait for addi
1	addi x1,x1,8	2	3		4	8	Commit in order
1	bne x2,x3,Loop	3	7			8	Wait for addi
2	ld x2,0(x1)	4	5	6	7	9	No execute delay
2	addi x2,x2,1	4	8		9	10	Wait for ld
2	sd x2,0(x1)	5	6			10	Wait for addi
2	addi x1,x1,8	5	6		7	11	Commit in order
2	bne x2,x3,Loop	6	10			11	Wait for addi
3	ld x2,0(x1)	7	8	9	10	12	Earliest possible
3	addi x2,x2,1	7	11		12	13	Wait for ld
3	sd x2,0(x1)	8	9			13	Wait for addi
3	addi x1,x1,8	8	9		10	14	Executes earlier
3	bne x2,x3,Loop	9	13			14	Wait for addi

Figure 3.24 The time of issue, execution, and writing result for a dual-issue version of our pipeline *with* speculation. Note that the ld following the bne can start execution early because it is speculative.

Advanced Techniques for Instruction Fetch, Issue, and Speculation

Dealing With Branches

1

Branch-Target Buffers

To reduce the branch penalty for our simple five-stage pipeline, as well as for deeper pipelines, we must know whether the as-yet-undecoded instruction is a branch and, if so, what the next program counter (PC) should be. If the instruction is a branch and we know what the next PC should be, we can have a branch penalty of zero. A branch-prediction cache that stores the predicted address for the next instruction after a branch is called a *branch-target buffer* or *branch-target cache.*

2

Branch Folding

One variation on the branch-target buffer is to store one or more *target instructions* instead of, or in addition to, the predicted *target address*. This variation has two potential advantages. First, it allows the branch-target buffer access to take longer than the time between successive instruction fetches, possibly allowing a larger branch-target buffer. Second, buffering the actual target instructions allows us to perform an optimization called *branch folding*. Branch folding can be used to obtain 0-cycle unconditional branches and sometimes 0-cycle conditional branches. As we will see, the Cortex A-53 uses a single-entry branch target cache that stores the predicted target instructions.

Procedure Return Address Buffer

Though procedure returns can be predicted with a branch-target buffer, the accuracy of such a prediction technique can be low if the procedure is called from multiple sites and the calls from one site are not clustered in time. For example, in SPEC CPU95, an aggressive branch predictor achieves an accuracy of less than 60% for such return branches. To overcome this problem, some designs use a small buffer of return addresses operating as a stack. This structure caches the most recent return addresses, pushing a return address on the stack at a call and popping one off at a return. If the cache is sufficiently large (i.e., as large as the maximum call depth), it will predict the returns perfectly. Figure 3.28 shows the performance of such a return buffer with 0–16 elements for a number of the SPEC CPU95 benchmarks. We will use a similar return predictor when we examine the studies of ILP in Section 3.10. Both the Intel Core processors and the AMD Phenom processors have return address predictors.

In large server applications, indirect jumps also occur for various function calls and control transfers. Predicting the targets of such branches is not as simple as in a procedure return. Some processors have opted to add specialized predictors for all indirect jumps, whereas others rely on a branch target buffer.

Although a simple predictor like gshare does a good job of predicting many conditional branches, it is not tailored to predicting loop branches, especially for long running loops. As we observed earlier, the Intel Core i7 920 used a specialized loop branch predictor. With the emergence of tagged hybrid predictors, which are as good at predicting loop branches, some recent designers have opted to put the resources into larger tagged hybrid predictors rather than a separate loop branch predictor.

Subroutine Return Stack - Diagram

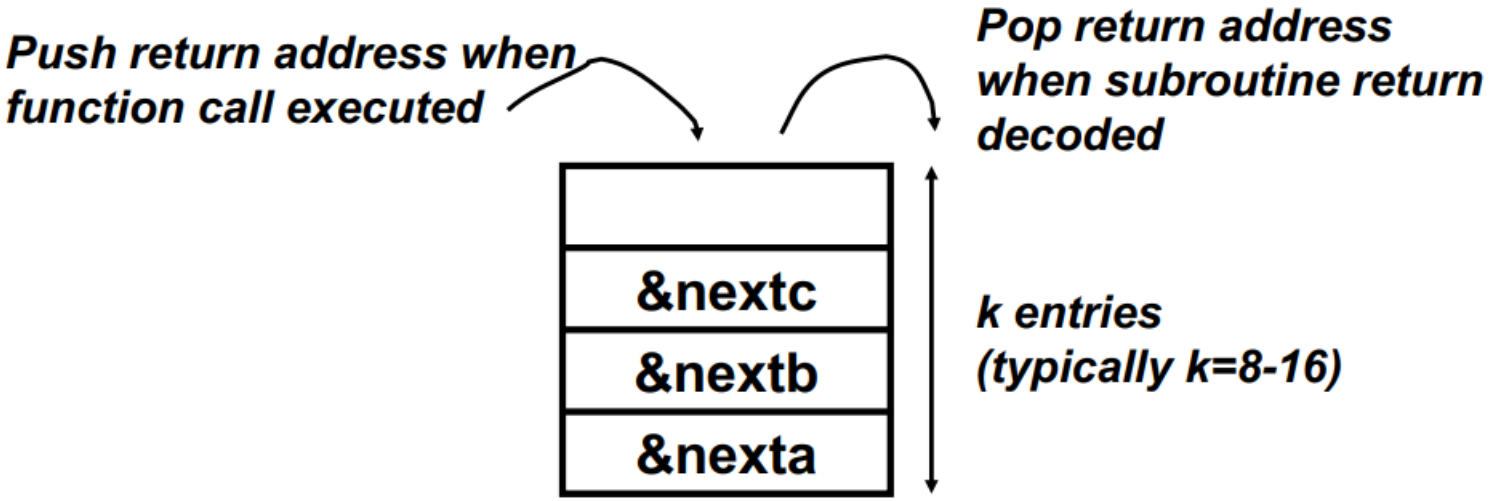
Subroutine Return Stack

Small structure to accelerate JR for subroutine returns, typically much more accurate than BTBs.

```
fa() { fb(); nexta: }

fb() { fc(); nextb: }

fc() { fd(); nextc: }
```



Integrated Instruction Fetch Unit

Instead, recent designs have used an integrated instruction fetch unit that integrates several functions:

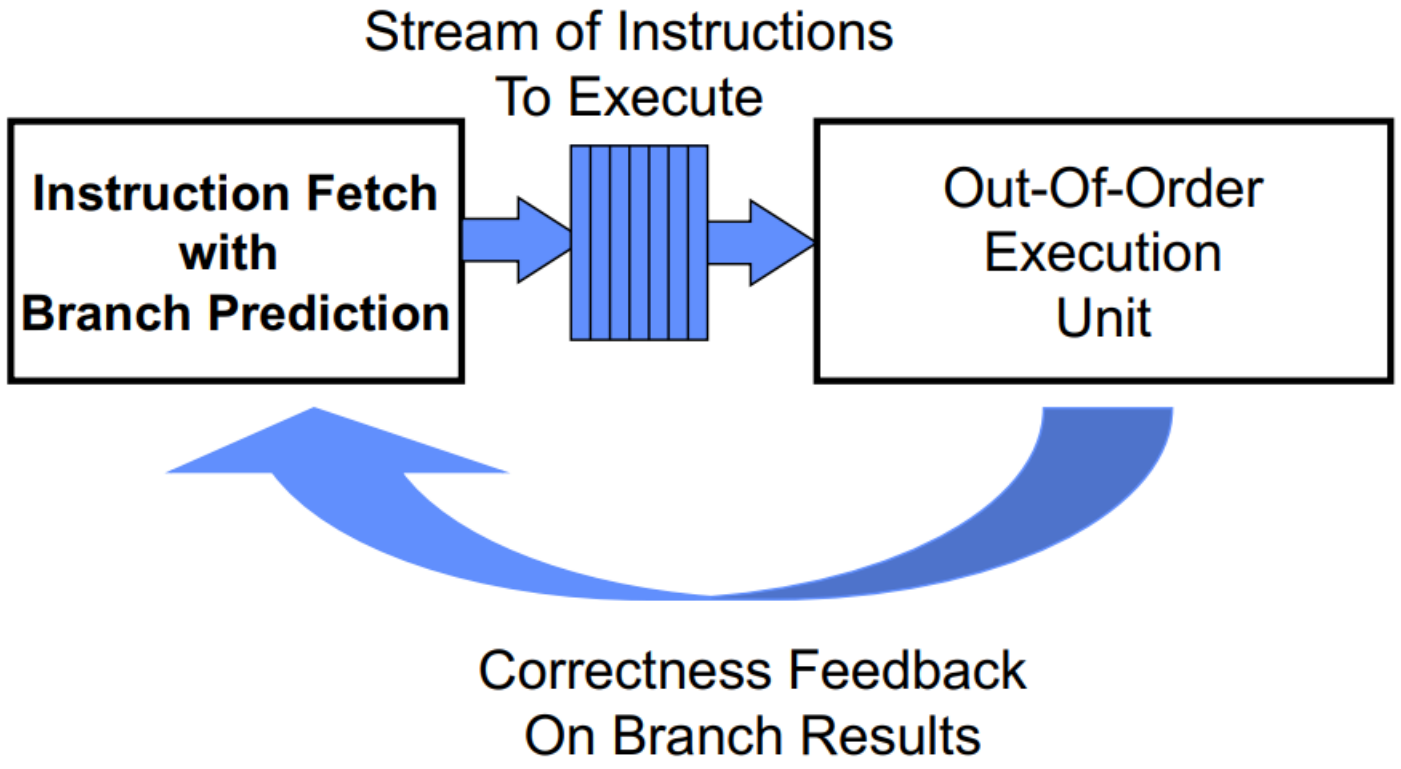
1. *Integrated branch prediction*—The branch predictor becomes part of the instruction fetch unit and is constantly predicting branches, so as to drive the fetch pipeline.
2. *Instruction prefetch*—To deliver multiple instructions per clock, the instruction fetch unit will likely need to fetch ahead. The unit autonomously manages the prefetching of instructions (see Chapter 2 for a discussion of techniques for doing this), integrating it with branch prediction.
3. *Instruction memory access and buffering*—When fetching multiple instructions per cycle, a variety of complexities are encountered, including the difficulty that fetching multiple instructions may require accessing multiple cache lines. The instruction fetch unit encapsulates this complexity, using prefetch to try to hide the cost of crossing cache blocks. The instruction fetch unit also provides buffering, essentially acting as an on-demand unit to provide instructions to the issue stage as needed and in the quantity needed.

Virtually all high-end processors now use a separate instruction fetch unit connected to the rest of the pipeline by a buffer containing pending instructions.

Independent Fetch Unit for Prefetching

Independent "Fetch" unit

- **Instruction fetch decoupled from execution**
  - **Instruction Buffer in-between**
- **Often issue logic (+ rename) included with Fetch**



Issuing Bundle of Instructions

Both register renaming and reorder buffers continue to be used in high-end processors, which now feature the ability to have as many as 100 or more instructions (including loads and stores waiting on the cache) in flight. Whether renaming or a reorder buffer is used, the key complexity bottleneck for a dynamically scheduled superscalar remains issuing bundles of instructions with dependences within the bundle. In particular, dependent instructions in an issue bundle must be issued with the assigned virtual registers of the instructions on which they depend. A strategy for instruction issue with register renaming similar to that used for multiple issue with reorder buffers (see page 205) can be deployed, as follows:

1. The issue logic reserves enough physical registers for the entire issue bundle (say, four registers for a four-instruction bundle with at most one register result per instruction).
2. The issue logic determines what dependences exist within the bundle. If a dependence does not exist within the bundle, the register renaming structure is used to determine the physical register that holds, or will hold, the result on which instruction depends. When no dependence exists within the bundle, the result is from an earlier issue bundle, and the register renaming table will have the correct register number.
3. If an instruction depends on an instruction that is earlier in the bundle, then the pre-reserved physical register in which the result will be placed is used to update the information for the issuing instruction.

Challenge of More Instruction Issues per Clock

Figure 3.29 shows a six-instruction code sequence and what the issue step must do. Remember that this must all occur in a single clock cycle, if the processor is to maintain a peak rate of six issues per clock! All the dependences must be detected, the physical registers must be assigned, and the instructions must be rewritten using the physical register numbers: in one clock. This example makes it clear why issue rates have grown from 3–4 to only 4–8 in the past 20 years. The complexity of the analysis required during the issue cycle grows as the square of the issue width, and a new processor is typically targeted to have a higher clock rate than in the last generation! Because register renaming and the reorder buffer approaches are duals, the same complexities arise independent of the implementation scheme.

How much to Speculate?

One of the significant advantages of speculation is its ability to uncover events that would otherwise stall the pipeline early, such as cache misses. This potential advantage, however, comes with a significant potential disadvantage. Speculation is not free. It takes time and energy, and the recovery of incorrect speculation further reduces performance. In addition, to support the higher instruction execution rate needed to benefit from speculation, the processor must have additional resources, which take silicon area and power. Finally, if speculation causes an exceptional event to occur, such as a cache or translation lookaside buffer (TLB) miss, the potential for significant performance loss increases, if that event would not have occurred without speculation.

To maintain most of the advantage while minimizing the disadvantages, most pipelines with speculation will allow only low-cost exceptional events (such as a first-level cache miss) to be handled in speculative mode. If an expensive exceptional event occurs, such as a second-level cache miss or a TLB miss, the processor will wait until the instruction causing the event is no longer speculative before handling the event. Although this may slightly degrade the performance of some programs, it avoids significant performance losses in others, especially those that suffer from a high frequency of such events coupled with less-than-excellent branch prediction.

In the 1990s the potential downsides of speculation were less obvious. As processors have evolved, the real costs of speculation have become more apparent, and the limitations of wider issue and speculation have been obvious. We return to this issue shortly.

Speculating through Multiple Branches

In the examples we have considered in this chapter, it has been possible to resolve a branch before having to speculate on another. Three different situations can benefit from speculating on multiple branches simultaneously: (1) a very high branch frequency, (2) significant clustering of branches, and (3) long delays in functional units. In the first two cases, achieving high performance may mean that multiple branches are speculated, and it may even mean handling more than one branch per clock. Database programs and other less structured integer computations, often exhibit these properties, making speculation on multiple branches important. Likewise, long delays in functional units can raise the importance of speculating on multiple branches as a way to avoid stalls from the longer pipeline delays.

Speculating on multiple branches slightly complicates the process of speculation recovery but is straightforward otherwise. As of 2017, no processor has yet combined full speculation with resolving multiple branches per cycle, and it is unlikely that the costs of doing so would be justified in terms of performance versus complexity and power.

Speculation and Energy Efficiency

What is the impact of speculation on energy efficiency? At first glance, one might argue that using speculation always decreases energy efficiency because whenever speculation is wrong, it consumes excess energy in two ways:

1. Instructions that are speculated and whose results are not needed generate excess work for the processor, wasting energy.
2. Undoing the speculation and restoring the state of the processor to continue execution at the appropriate address consumes additional energy that would not be needed without speculation.

Certainly, speculation will raise the power consumption, and if we could control speculation, it would be possible to measure the cost (or at least the dynamic power cost). But, if speculation lowers the execution time by more than it increases the average power consumption, then the total energy consumed may be less.

Thus, to understand the impact of speculation on energy efficiency, we need to look at how often speculation is leading to unnecessary work. If a significant number of unneeded instructions is executed, it is unlikely that speculation will improve running time by a comparable amount. Figure 3.30 shows the fraction of instructions that are executed from misspeculation for a subset of the SPEC2000 benchmarks using a sophisticated branch predictor. As we can see, this fraction of executed misspeculated instructions is small in scientific code and significant (about 30% on average) in integer code. Thus it is unlikely that speculation is energy-efficient for integer applications, and the end of Dennard scaling makes imperfect speculation more problematic. Designers could avoid speculation, try to reduce the misspeculation, or think about new approaches, such as only speculating on branches that are known to be highly predictable.

Explicit Register Renaming Process

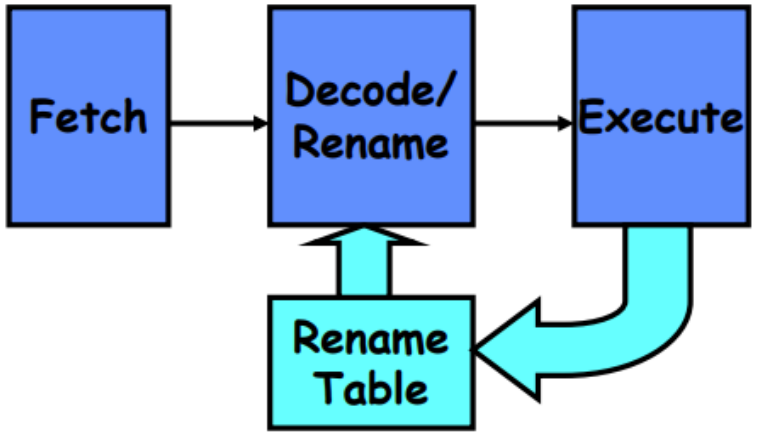
Explicit Register Renaming

- Tomasulo provides *Implicit Register Renaming*
  - User registers renamed to reservation station tags
- Explicit Register Renaming:
  - Use *physical* register file that is larger than number of registers specified by ISA
- Keep a translation table:
  - ISA register => physical register mapping
  - When register is written, replace table entry with new register from freelist.
  - Physical register becomes free when not being used by any instructions in progress.
- Pipeline can be exactly like "standard" DLX pipeline
  - IF, ID, EX, etc....
- Advantages:
  - Removes all WAR and WAW hazards
  - Like Tomasulo, good for allowing full out-of-order completion
  - Allows data to be fetched from a single register file
  - Makes speculative execution/precise interrupts easier:
    - » All that needs to be "undone" for precise break point is to undo the table mappings

Register Renaming Architecture

Explicit Register Renaming

- Make use of a *physical* register file that is larger than number of registers specified by ISA
- Keep a translation table:
  - ISA register => physical register mapping
  - When register is written, replace table entry with new register from freelist.
  - Physical register becomes free when not being used by any instructions in progress.



Questions and Answers

- Solved Examples
  - VLIW Eg 1
  - VLIW Eg 2
  - VLIW Eg 3

Cross-Cutting Issues

Cross-Cutting Issues

Hardware Speculation
Vs
Software Speculation

Hardware Versus Software Speculation

The hardware-intensive approaches to speculation in this chapter and the software approaches of Appendix H provide alternative approaches to exploiting ILP. Some of the trade-offs, and the limitations, for these approaches are listed here:

- To speculate extensively, we must be able to disambiguate memory references. This capability is difficult to do at compile time for integer programs that contain pointers. In a hardware-based scheme, dynamic runtime disambiguation of memory addresses is done using the techniques we saw earlier for Tomasulo's algorithm. This disambiguation allows us to move loads past stores at runtime. Support for speculative memory references can help overcome the conservatism of the compiler, but unless such approaches are used carefully, the overhead of the recovery mechanisms may swamp the advantages.
- Hardware-based speculation works better when control flow is unpredictable and when hardware-based branch prediction is superior to software-based branch prediction done at compile time. These properties hold for many integer programs, where the misprediction rates for dynamic predictors are usually less than one-half of those for static predictors. Because speculated instructions may slow down the computation when the prediction is incorrect, this difference is significant. One result of this difference is that even statically scheduled processors normally include dynamic branch predictors.
- Hardware-based speculation maintains a completely precise exception model even for speculated instructions. Recent software-based approaches have added special support to allow this as well.
- Hardware-based speculation does not require compensation or bookkeeping code, which is needed by ambitious software speculation mechanisms.
- Compiler-based approaches may benefit from the ability to see further into the code sequence, resulting in better code scheduling than a purely hardware-driven approach.

Issues - continued

Hardware Speculation
Vs
Software Speculation

- Hardware-based speculation with dynamic scheduling does not require different code sequences to achieve good performance for different implementations

of an architecture. Although this advantage is the hardest to quantify, it may be the most important one in the long run. Interestingly, this was one of the motivations for the IBM 360/91. On the other hand, more recent explicitly parallel architectures, such as IA-64, have added flexibility that reduces the hardware dependence inherent in a code sequence.

The major disadvantage of supporting speculation in hardware is the complexity and additional hardware resources required. This hardware cost must be evaluated against both the complexity of a compiler for a software-based approach and the amount and usefulness of the simplifications in a processor that relies on such a compiler.

Some designers have tried to combine the dynamic and compiler-based approaches to achieve the best of each. Such a combination can generate interesting and obscure interactions. For example, if conditional moves are combined with register renaming, a subtle side effect appears. A conditional move that is annulled must still copy a value to the destination register because it was renamed earlier in the instruction pipeline. These subtle interactions complicate the design and verification process and can also reduce performance.

The Intel Itanium processor was the most ambitious computer ever designed based on the software support for ILP and speculation. It did not deliver on the hopes of the designers, especially for general-purpose, nonscientific code. As designers' ambitions for exploiting ILP were reduced in light of the difficulties described on page 244, most architectures settled on hardware-based mechanisms with issue rates of three to four instructions per clock.

Speculative Execution and the Memory System

Inherent in processors that support speculative execution or conditional instructions is the possibility of generating invalid addresses that would not occur without speculative execution. Not only would this be incorrect behavior if protection exceptions were taken, but also the benefits of speculative execution would be swamped by false exception overhead. Therefore the memory system must identify speculatively executed instructions and conditionally executed instructions and suppress the corresponding exception.

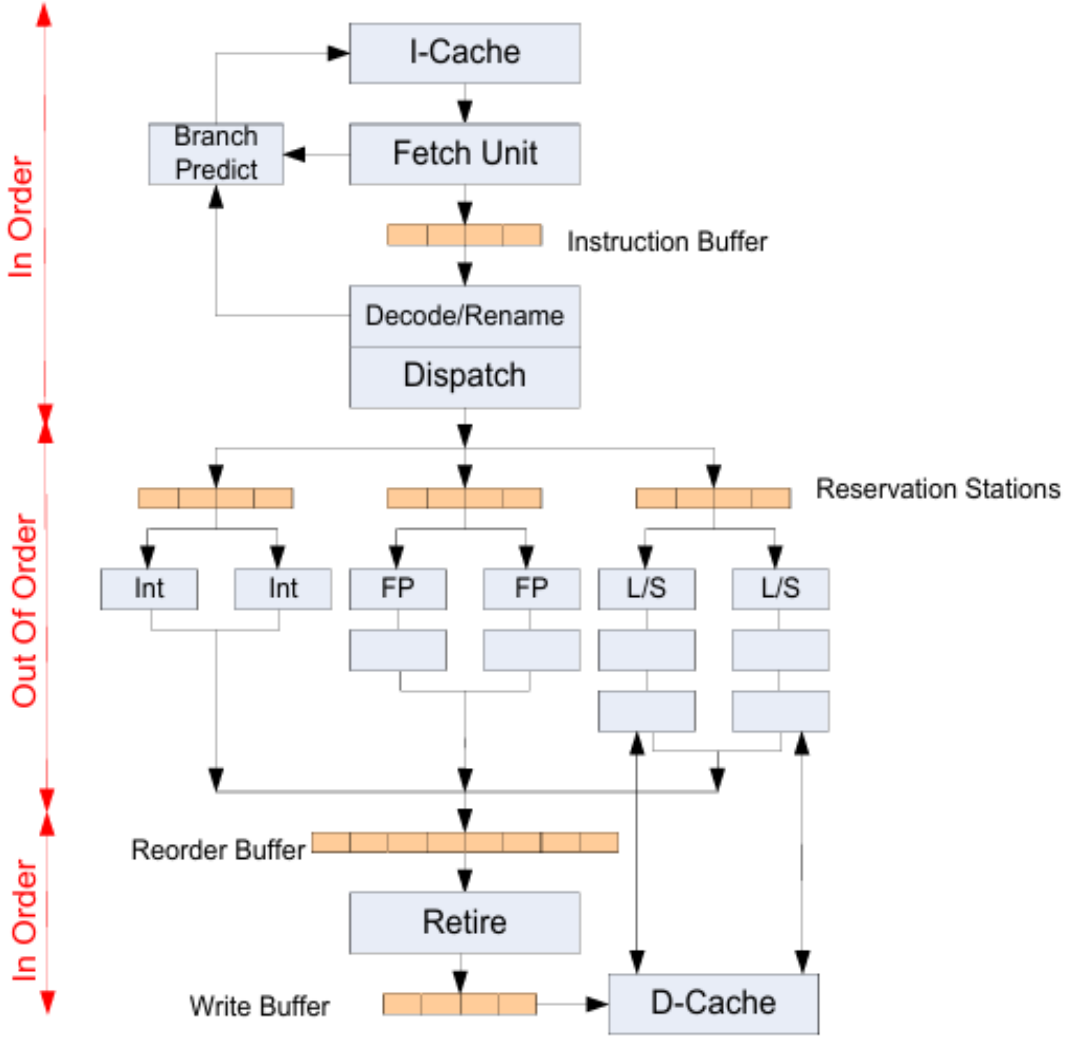
By similar reasoning, we cannot allow such instructions to cause the cache to stall on a miss because, again, unnecessary stalls could overwhelm the benefits of speculation. Thus these processors must be matched with nonblocking caches.

In reality, the penalty of a miss that goes to DRAM is so large that speculated misses are handled only when the next level is on-chip cache (L2 or L3). Figure 2.5 on page 84 shows that for some well-behaved scientific programs, the compiler can sustain multiple outstanding L2 misses to cut the L2 miss penalty effectively. Once again, for this to work, the memory system behind the cache must match the goals of the compiler in number of simultaneous memory accesses.

Putting It All Together: The Intel Core i7 6700 and ARM Cortex-A53 247

(Memory System to be discussed later, after midterm)

A Modern out-of-order Superscalar Processor



Reconstruct dataflow graph

Execute each instruction as soon as its source operands are available

Write back results in program order

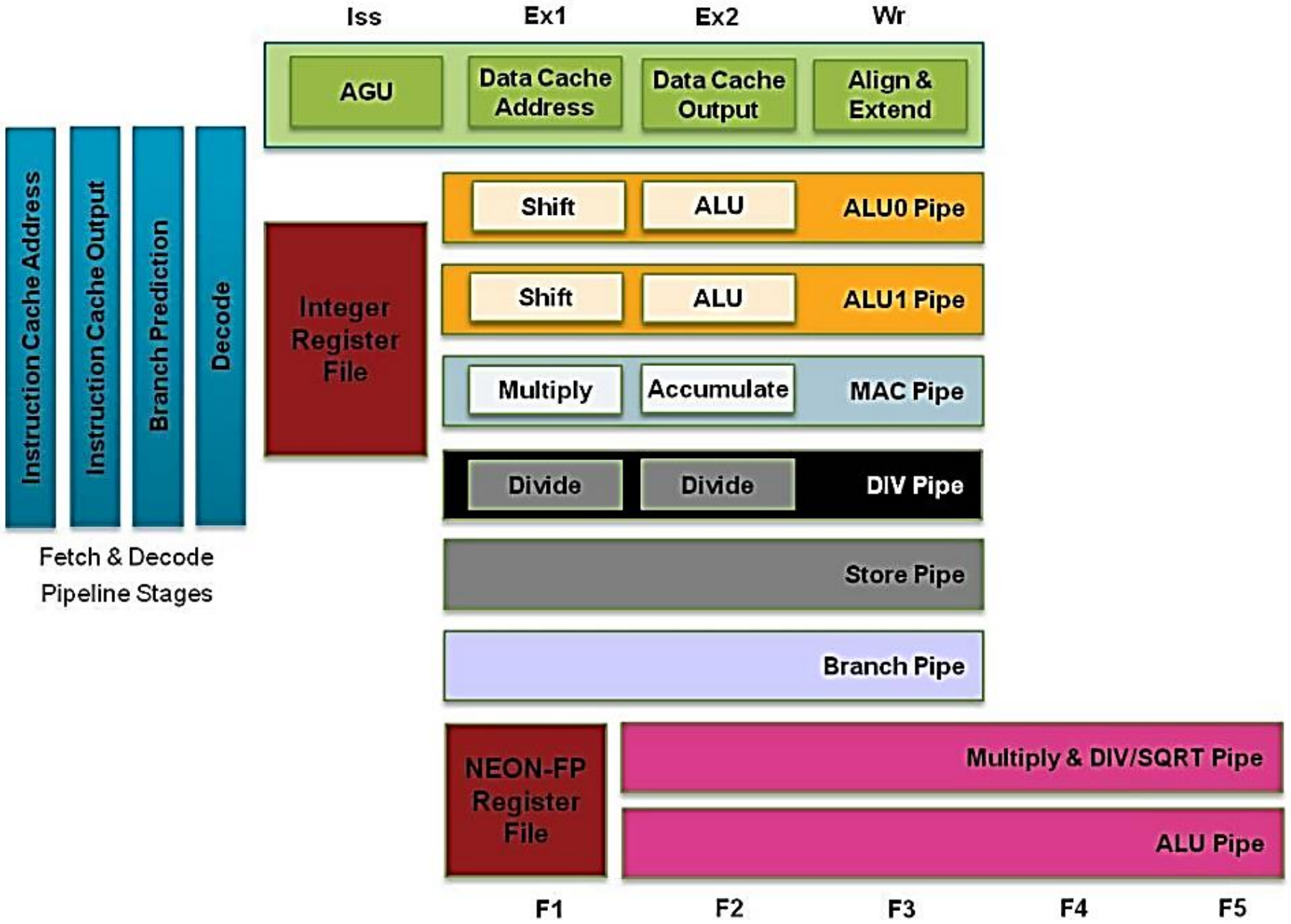
Why is this needed?

ARM Cortex A53 Architecture and Performance

Cortex A53

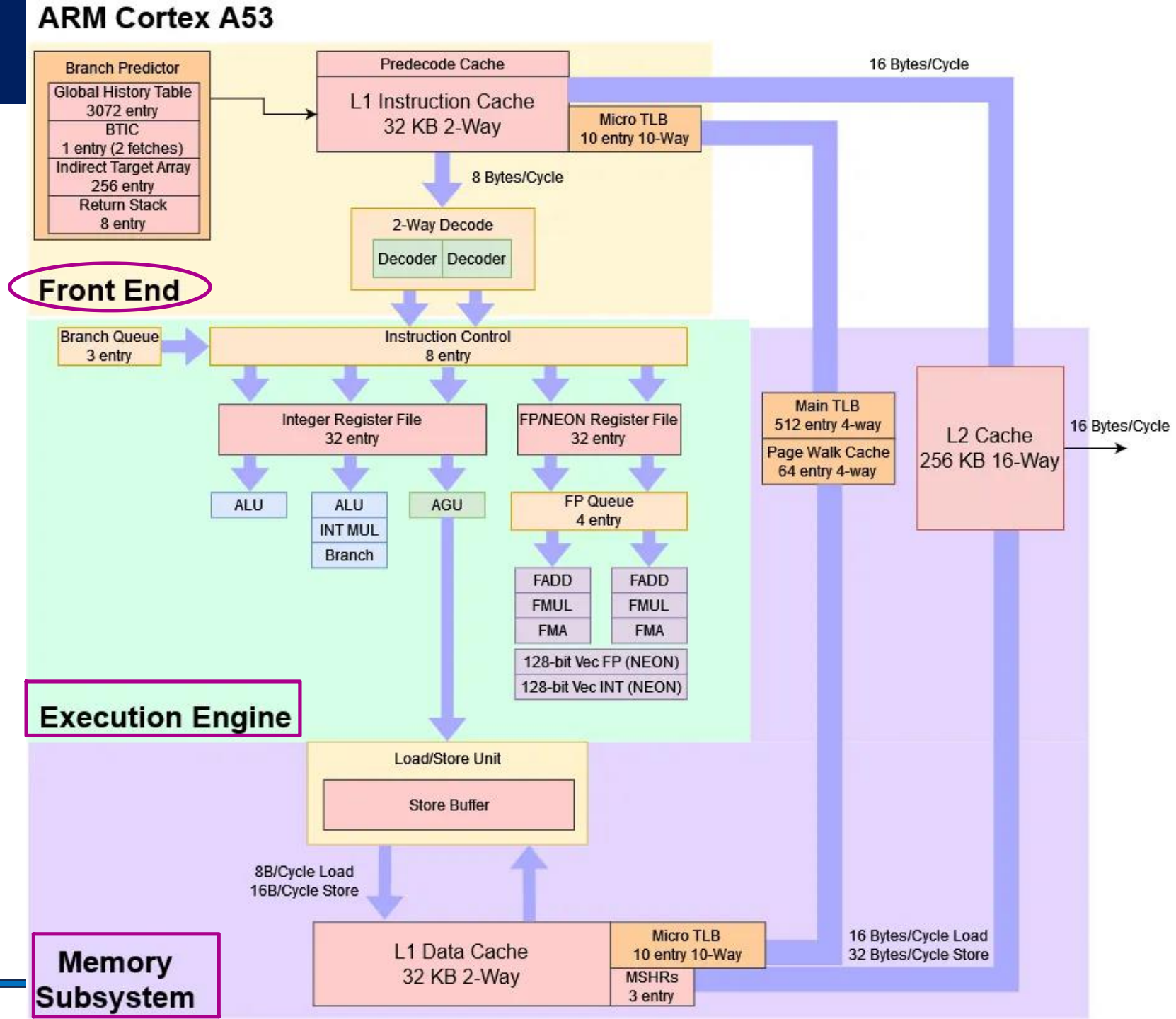
The Cortex-A53 pipeline is 8-stages deep for integer instructions and 10-stages deep for floating-point and NEON instructions. Figure 3-1 illustrates the structure of the datapath:

Block Level Diagram



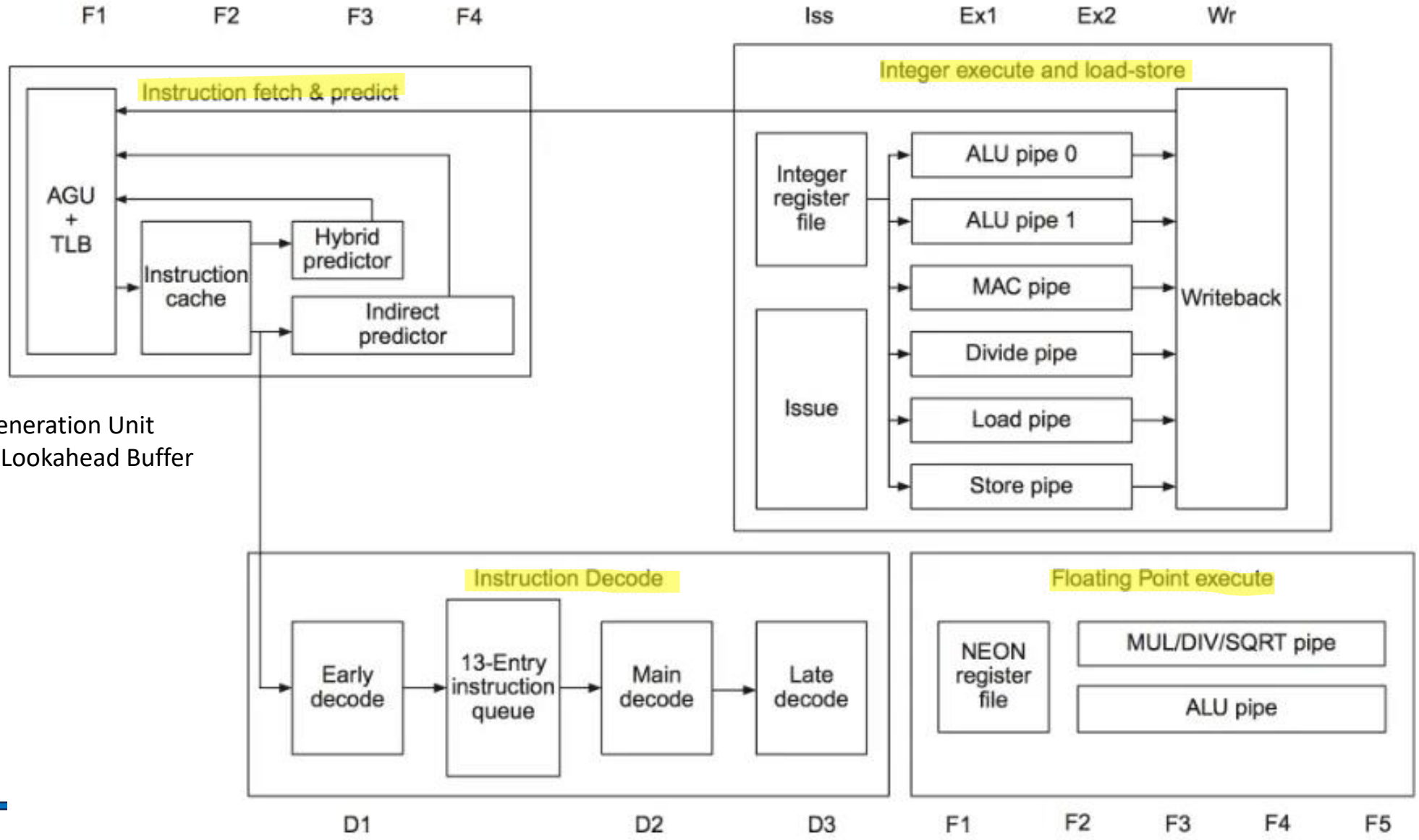
Cortex A53

Architecture Details



ARM Cortex A53 Architecture

– **Dual-issue, statically scheduled superscalar with dynamic issue detection → 0.5 CPI ideally**



AGU = Address Generation Unit
TLB = Translation Lookahead Buffer

A53 Integer Pipeline

- The basic structure of the A53 integer pipeline is 8 stages:
- F1 and F2 fetch the instruction,
- D1 and D2 do the basic decoding, and D3 decodes some more complex instructions and is overlapped with the first stage of the execution pipeline (ISS).
- After ISS, the Ex1, EX2, and WB stages complete the integer pipeline.
- Branches use four different predictors, depending on the type.
- The floating-point execution pipeline is 5 cycles deep, in addition to the 5 cycles needed for fetch and decode, yielding 10 stages in total.

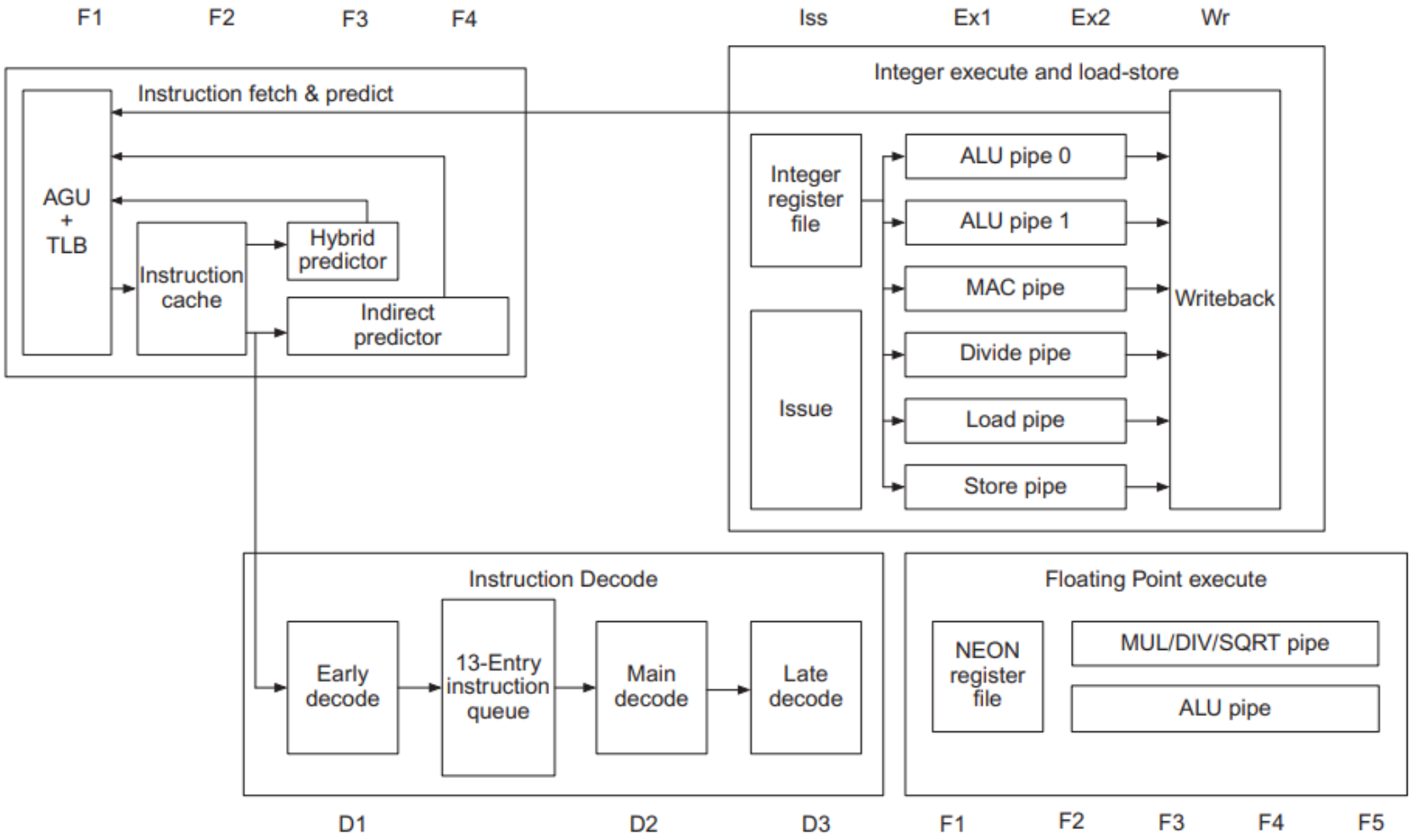


Figure 3.34 The basic structure of the A53 integer pipeline is 8 stages: F1 and F2 fetch the instruction, D1 and D2 do the basic decoding, and D3 decodes some more complex instructions and is overlapped with the first stage of the execution pipeline (ISS). After ISS, the Ex1, EX2, and WB stages complete the integer pipeline. Branches use four different predictors, depending on the type. The floating-point execution pipeline is 5 cycles deep, in addition to the 5 cycles needed for fetch and decode, yielding 10 stages in total.

Architecture Overview ARM Cortex A53

- 8-stage pipelined processor with 2-way superscalar, in-order execution pipeline
- DSP and NEON SIMD extensions are mandatory per core
- VFPv4 Floating Point Unit onboard (per core)
- Hardware virtualization support
- TrustZone security extensions
- 64-byte cache lines
- 10-entry L1 TLB, and 512-entry L2 TLB
- 4 KiB conditional branch predictor, 256-entry indirect branch predictor

Architecture Details of A53

F3, F4 are variants
Of microarchitecture

The four cycles of instruction fetch include an address generation unit that produces the next PC either by incrementing the last PC or from one of four predictors:

1. A single-entry branch target cache containing two instruction cache fetches (the next two instructions following the branch, assuming the prediction is correct). This target cache is checked during the first fetch cycle, if it hits; then the next two instructions are supplied from the target cache. In case of a hit and a correct prediction, the branch is executed with no delay cycles.
2. A 3072-entry hybrid predictor, used for all instructions that do not hit in the branch target cache, and operating during F3. Branches handled by this predictor incur a 2-cycle delay.
3. A 256-entry indirect branch predictor that operates during F4; branches predicted by this predictor incur a three-cycle delay when predicted correctly.
4. An 8-deep return stack, operating during F4 and incurring a three-cycle delay.

Branch decisions are made in ALU pipe 0, resulting in a branch misprediction penalty of 8 cycles. Figure 3.35 shows the misprediction rate for SPECint2006. The amount of work that is wasted depends on both the misprediction rate and the issue rate sustained during the time that the mispredicted branch was followed. As Figure 3.36 shows, wasted work generally follows the misprediction rate, though it may be larger or occasionally shorter.

Performance of A53 Pipeline Stalls

Performance of the A53 Pipeline

The A53 has an ideal CPI of 0.5 because of its dual-issue structure. Pipeline stalls can arise from three sources:

1. Functional hazards, which occur because two adjacent instructions selected for issue simultaneously use the same functional pipeline. Because the A53 is statically scheduled, the compiler should try to avoid such conflicts. When such instructions appear sequentially, they will be serialized at the beginning of the execution pipeline, when only the first instruction will begin execution.
2. Data hazards, which are detected early in the pipeline and may stall either both instructions (if the first cannot issue, the second is always stalled) or the second of a pair. Again, the compiler should try to prevent such stalls when possible.
3. Control hazards, which arise only when branches are mispredicted.

Both TLB misses and cache misses also cause stalls. On the instruction side, a TLB or cache miss causes a delay in filling the instruction queue, likely leading to a downstream stall of the pipeline. Of course, this depends on whether it is an L1 miss, which might be largely hidden if the instruction queue was full at the time of the miss, or an L2 miss, which takes considerably longer. On the data side, a cache or TLB miss will cause the pipeline to stall because the load or store that caused the miss cannot proceed down the pipeline. All other subsequent instructions will thus be stalled. Figure 3.37 shows the CPI and the estimated contributions from various sources.

The A53 uses a shallow pipeline and a reasonably aggressive branch predictor, leading to modest pipeline losses, while allowing the processor to achieve high clock rates at modest power consumption. In comparison with the i7, the A53 consumes approximately 1/200 the power for a quad core processor!

Cortex A53 Pipeline Architecture and Stages

The Cortex-A53 processor, a member of ARM's Cortex-A series, is designed with an 8-stage pipeline to balance performance and power efficiency. The pipeline stages are meticulously crafted to handle instruction processing with minimal stalls and maximum throughput. Each stage has a specific role in the instruction execution process, and understanding these stages is crucial for optimizing software and diagnosing performance bottlenecks.

The Cortex-A53 pipeline can be broadly divided into three phases: Front-End, Execution, and Back-End. The Front-End is responsible for fetching and decoding instructions, the Execution phase handles the actual computation, and the Back-End manages memory access and write-back operations. Below is a detailed breakdown of the 8 pipeline stages:

Fetch Stage 1 (F1): The first stage of the pipeline is responsible for fetching instructions from the instruction cache (I-cache) or memory. The program counter (PC) is used to determine the address of the next instruction to be fetched. The Cortex-A53 employs branch prediction to minimize stalls caused by control flow changes.

Fetch Stage 2 (F2): In this stage, the fetched instructions are aligned and prepared for decoding. The Cortex-A53 can fetch up to two instructions per cycle, depending on the alignment and availability of instructions in the I-cache.

Cortex A53 Pipeline Architecture and Stages

Decode Stage 1 (D1): The first decode stage begins the process of translating the fetched instructions into micro-operations (µOps). This stage identifies the instruction type and prepares it for further decoding.
Decode Stage 2 (D2): The second decode stage completes the translation of instructions into µOps. The Cortex-A53 uses a dual-issue pipeline, meaning it can decode and issue up to two instructions per cycle to the execution units.

Issue Stage (IS): The issue stage is responsible for dispatching the decoded µOps to the appropriate execution units. The Cortex-A53 has multiple execution units, including integer ALUs, floating-point units, and load/store units. The issue stage ensures that dependencies are resolved and that instructions are issued in the correct order.

Execute Stage 1 (E1): The first execute stage performs the actual computation or operation specified by the instruction. This stage includes arithmetic logic unit (ALU) operations, address generation for memory access, and other computational tasks.

Execute Stage 2 (E2): The second execute stage completes any remaining computation and prepares the results for write-back. For memory operations, this stage includes data cache (D-cache) access and alignment.

Write-Back Stage (WB): The final stage of the pipeline writes the results of the executed instructions back to the register file or memory. This stage ensures that the architectural state of the processor is updated correctly.

Load/Store Unit in Cortex A53 (Memory – later)

- A CPU's load/store unit has a very complex job, especially on out-of-order CPUs where it has to track a lot of memory operations in flight and make them appear to execute in program order. The A53's load/store unit is simpler thanks to in-order execution, but handling memory operations still isn't easy.
- It still has to handle instructions that address memory with byte granularity and varying sizes, with a L1 data cache that accesses a larger, aligned region under the hood. It still has to handle memory dependencies. Even though the A53 is in-order, pipelining means a load can need data from a store that hasn't committed yet.
- The A53 appears to access its data cache in 8 byte (64-bit) aligned chunks. Accesses that span a 8-byte boundary incur a 1 cycle penalty. Forwarding similarly incurs a 1 cycle penalty, but notably, there is no massively higher cost if a load only partially overlaps a store.
- In the worst case, forwarding costs 6 cycles when the load partially overlaps the store and the load address is higher than the store address.

ARM Branch Predictor

A53 is a particularly interesting case both because it targets very low power and area, and because branch prediction accuracy matters less for it. That's because A53 can't speculate as far as out-of-order CPUs.

ARM has designed a tiny branch predictor that emphasizes low power and area over speed and accuracy. The A53 Technical Reference Manual states that the branch predictor has a global history table with 3072 entries. For comparison, AMD's Athlon 64 from 2003 had a 16384 entry history table.



BTIC = Branch Target Instruction Cache

Forwarding and Dual-Issue

3.1.1 Forwarding Paths

Forwarding paths are implemented between almost all integer pipeline stages where operands can be consumed. For example:

- The ALU pipeline can forward results from the end of Ex1 and Ex2 to earlier stages of both of the the ALU pipelines and to the Iss stage of the divide and MAC pipelines
- The Divide pipeline can only accept operands in the Iss stage and will only forward results from the Wr stage
- The MAC pipeline can only accept multiply operands in the Iss stage and the accumulate operand in the Ex2 stage. There is a dedicated forwarding path from the Wr stage to the Ex2 stage for accumulator forwarding within the MAC pipeline. Multiply and MAC results can be forwarded from the Wr stage.
- Load-Store instructions require their address operands at the beginning of the Iss stage. More details are provided in [3].
- With the exception of system register read results, all integer results can be forwarded from the Wr stage.

Forwarding does not contain bubbles so if a result can be forwarded from the end of the Ex1 stage it can also be forwarded from the Ex2 and Wr stages. Similarly, if the latest a result can be consumed is Ex2, it can also be consumed in Ex1 or Iss if the result is available earlier.

Forwarding in the FP-NEON pipelines is more complex and dependent on whether the instruction is traversing the Multiply & DIV/SQRT pipeline or the ALU pipeline. More details are provided in [3].

Due to the number of forwarding paths there is no further description in this document. Instead, please refer to [3] for individual instruction behaviour.

3.2 Dual-Issue

Cortex-A53 can dual-issue under most circumstances.

Detailed information about dual-issue opportunities are described in [3]. However an outline of the rules required to achieve dual-issue are described in Table 3-1 and Table 3-2. In these tables instruction-0 is the instruction that would otherwise be single-issued (also known as the older instruction) and instruction-1 (also known as the younger instruction) can only dual-issue if instruction-0 also supports dual-issue.

Return Prediction Stack

Return Prediction

For returns, the A53 has an eight-deep return stack. This is quite shallow compared to other cores. For example, Kryo has a 16-deep return stack. AMD's Athlon 64's return stack had 12 entries. But even an eight-deep return stack is better than nothing, and could be adequate for the majority of cases.



Integer Execution Units

A53 can dual-issue the most common operations like integer adds, register to register MOVs, and bitwise operations. Less common operations like integer multiplies and branches can't be dual issued but can issue alongside another type of instruction.

Instruction	IPC	Latency
Integer Add	1.77	1 cycle
Branch (Not Taken)	0.95	See branch prediction section above
64-bit Integer Multiply	0.95	4 cycles
64-bit Load from Memory	1	See caching section below

Floating Point and Vector Execution

- Unlike the original Pentium, which only had one FP pipe, the A53 uses modern transistor budgets to dual issue common floating-point instructions like FP adds and multiplies.
- That's a nice capability to have because Javascript's numbers are all floating point, and A53 may have to deal with a lot of Javascript in phones.
- Unfortunately, latency is 4 cycles, which isn't good considering the A53's low clock speed. It's even worse because the A53 doesn't benefit from a big out-of-order engine, which means the core must stall to resolve execution dependencies.

Instruction	IPC	Latency
FP32 Add	1.82	4 cycles
FP32 Multiply	1.67	4 cycles
FP32 FMA	1.19	8 cycles
128-bit Vector FP32 Add	0.91	4 cycles
128-bit Vector FP32 Multiply	0.91	4 cycles
128-bit Vector FP32 FMA	0.95	8 cycles
128-bit Vector INT32 Add	0.95	2 cycles
128-bit Vector INT32 Multiply	0.91	4 cycles
128-bit Load from Memory	0.49	Not tested
128-bit Store	1	N/A

FMA = Fused Multiply Add

Misprediction

ARM Cortex-A53 Misprediction Rate

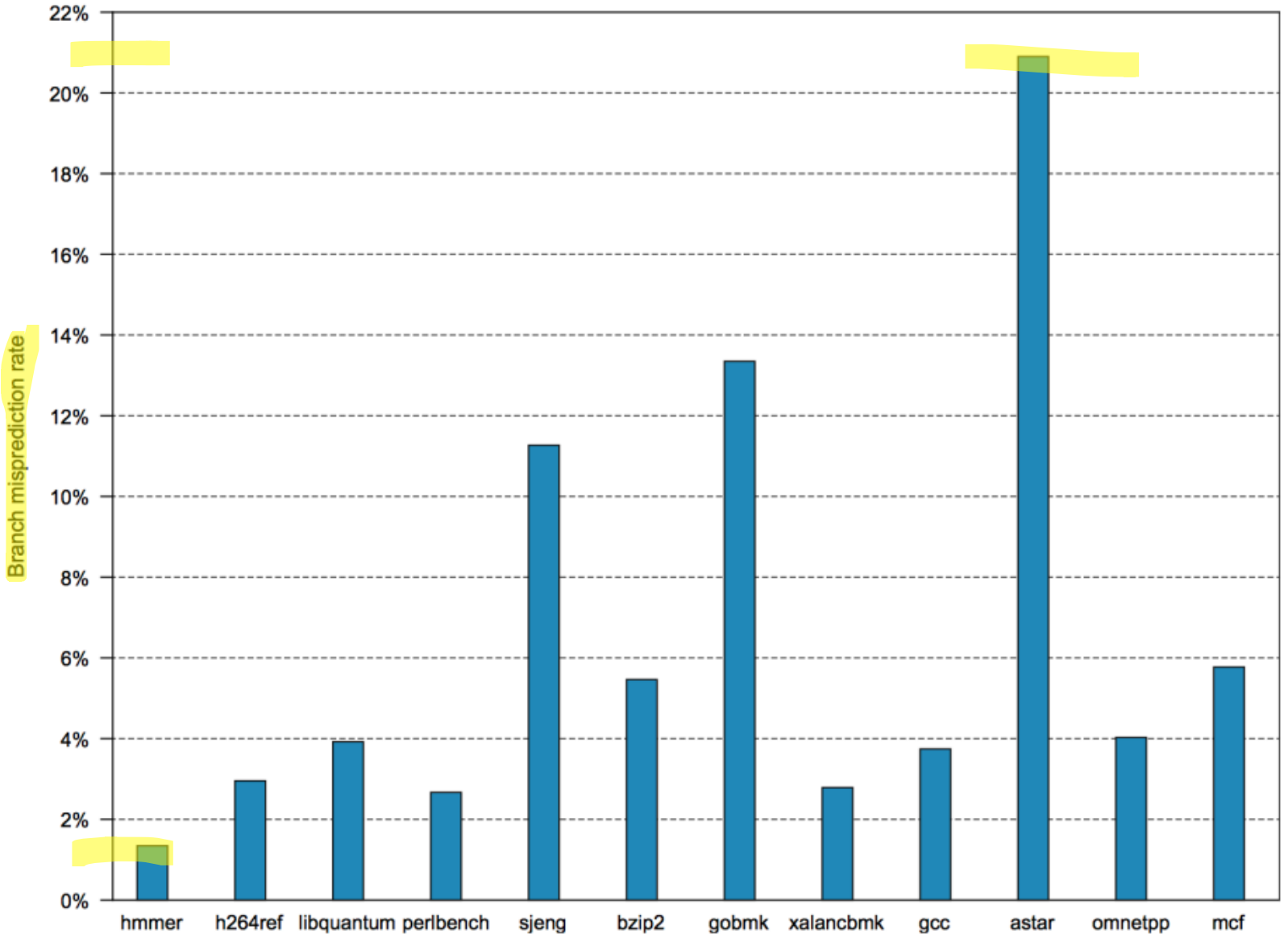


Figure 3.35 Misprediction rate of the A53 branch predictor for SPECint2006.

CPI Analysis

Estimated Composition of ARM A53 CPI

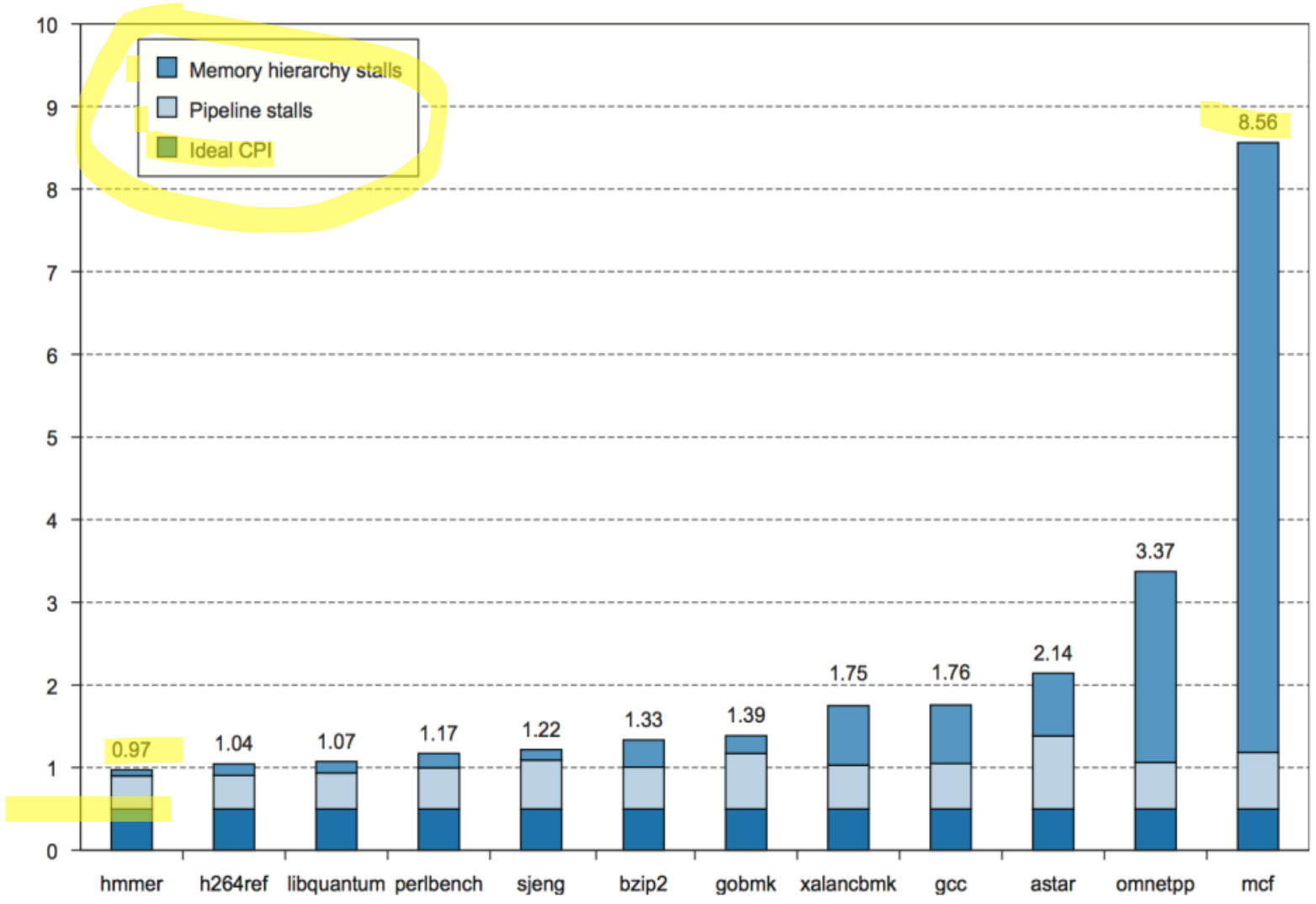


Figure 3.37 The estimated composition of the CPI on the ARM A53 shows that pipeline stalls are significant but are outweighed by cache misses in the poorest performing programs. This estimate is obtained by using the L1 and L2 miss rates and penalties to compute the L1 and L2 generated stalls per instruction. These are subtracted from the CPI measured by a detailed simulator to obtain the pipeline stalls. Pipeline stalls include all three hazards.

Architecture Details of A53

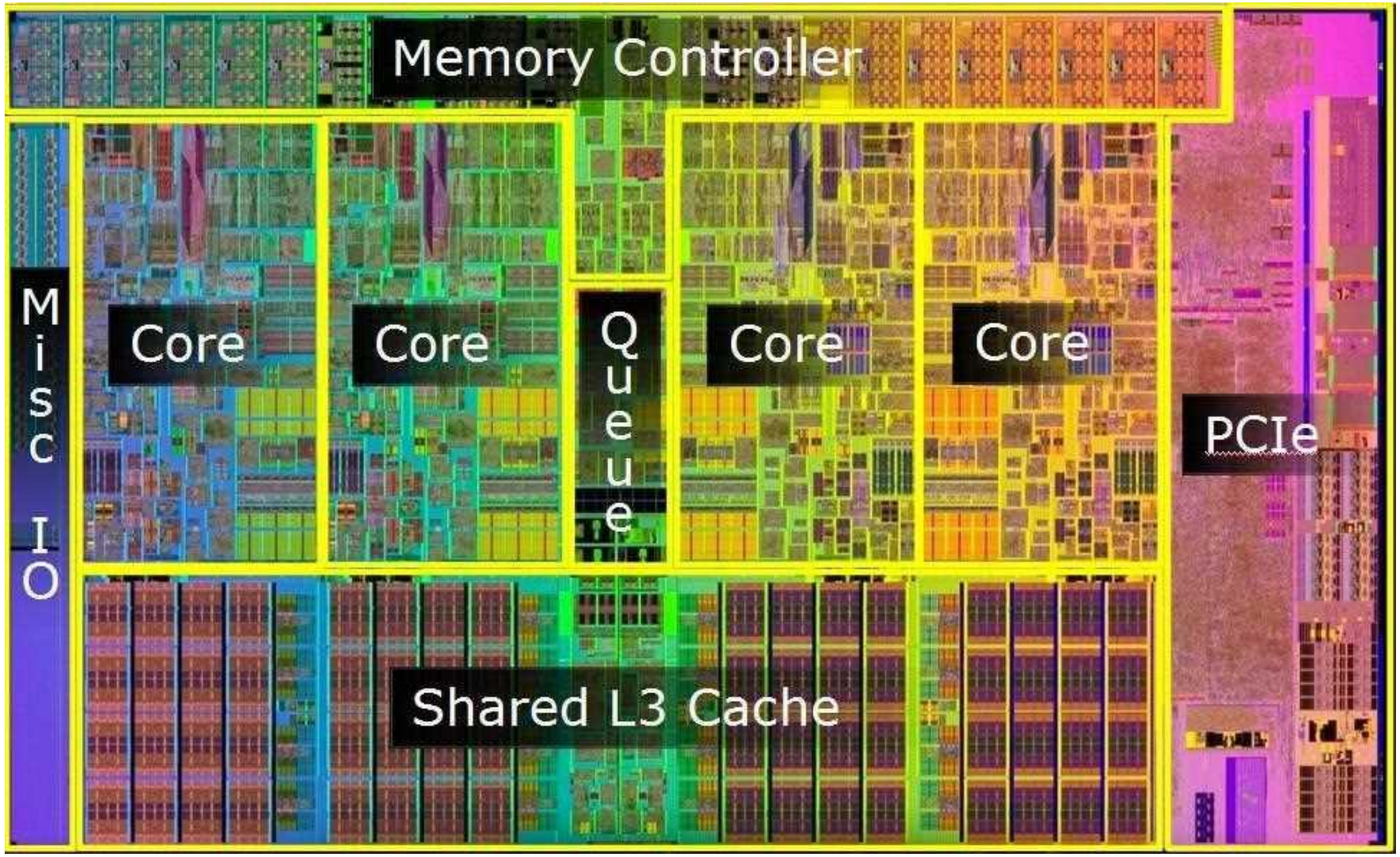
The four cycles of instruction fetch include an address generation unit that produces the next PC either by incrementing the last PC or from one of four predictors:

1. A single-entry branch target cache containing two instruction cache fetches (the next two instructions following the branch, assuming the prediction is correct). This target cache is checked during the first fetch cycle, if it hits; then the next two instructions are supplied from the target cache. In case of a hit and a correct prediction, the branch is executed with no delay cycles.
2. A 3072-entry hybrid predictor, used for all instructions that do not hit in the branch target cache, and operating during F3. Branches handled by this predictor incur a 2-cycle delay.
3. A 256-entry indirect branch predictor that operates during F4; branches predicted by this predictor incur a three-cycle delay when predicted correctly.
4. An 8-deep return stack, operating during F4 and incurring a three-cycle delay.

Branch decisions are made in ALU pipe 0, resulting in a branch misprediction penalty of 8 cycles. Figure 3.35 shows the misprediction rate for SPECint2006. The amount of work that is wasted depends on both the misprediction rate and the issue rate sustained during the time that the mispredicted branch was followed. As Figure 3.36 shows, wasted work generally follows the misprediction rate, though it may be larger or occasionally shorter.

Intel Core i7 6700

Chip view – Intel core i7 processor



Core i7 Pipeline

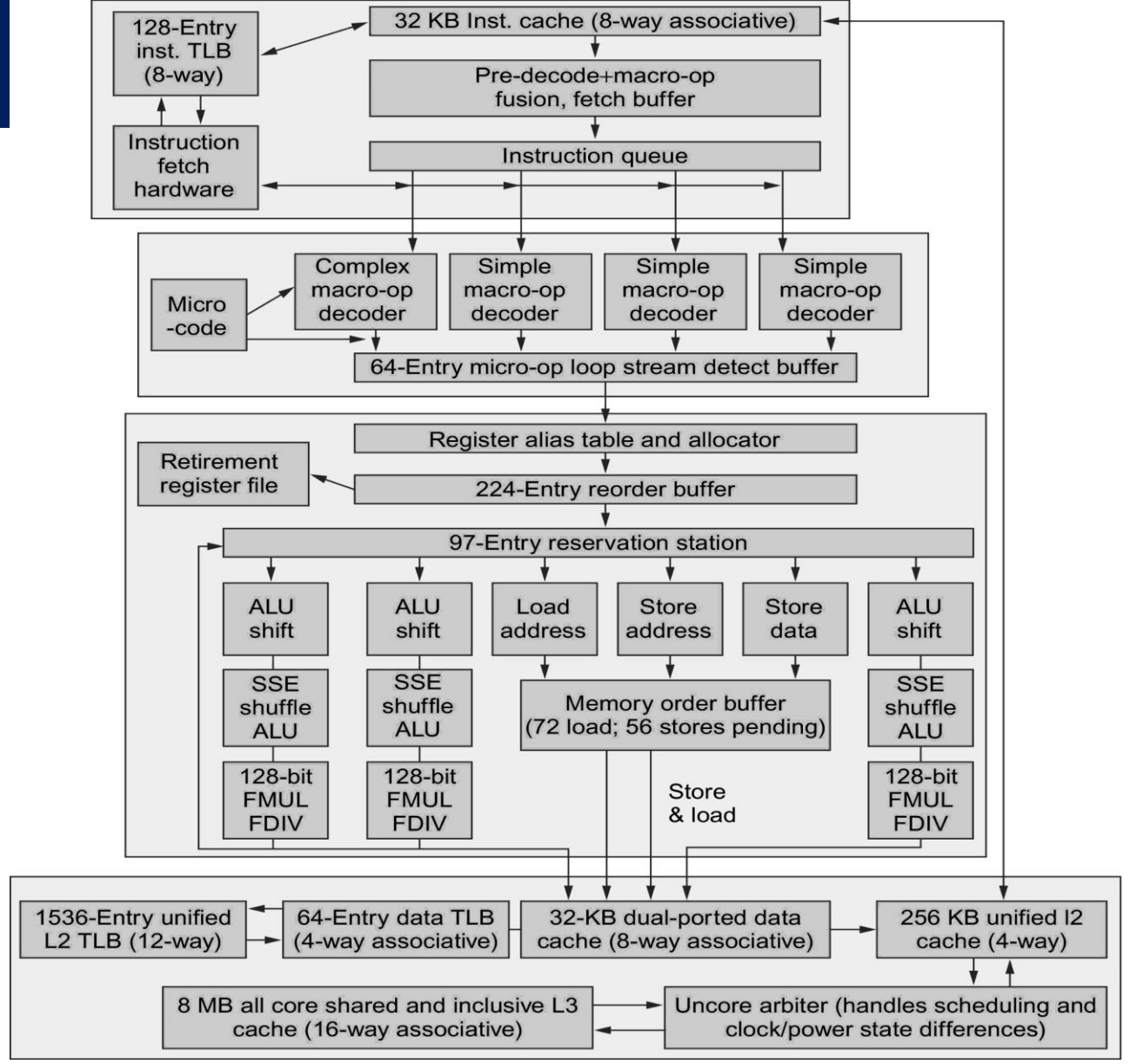
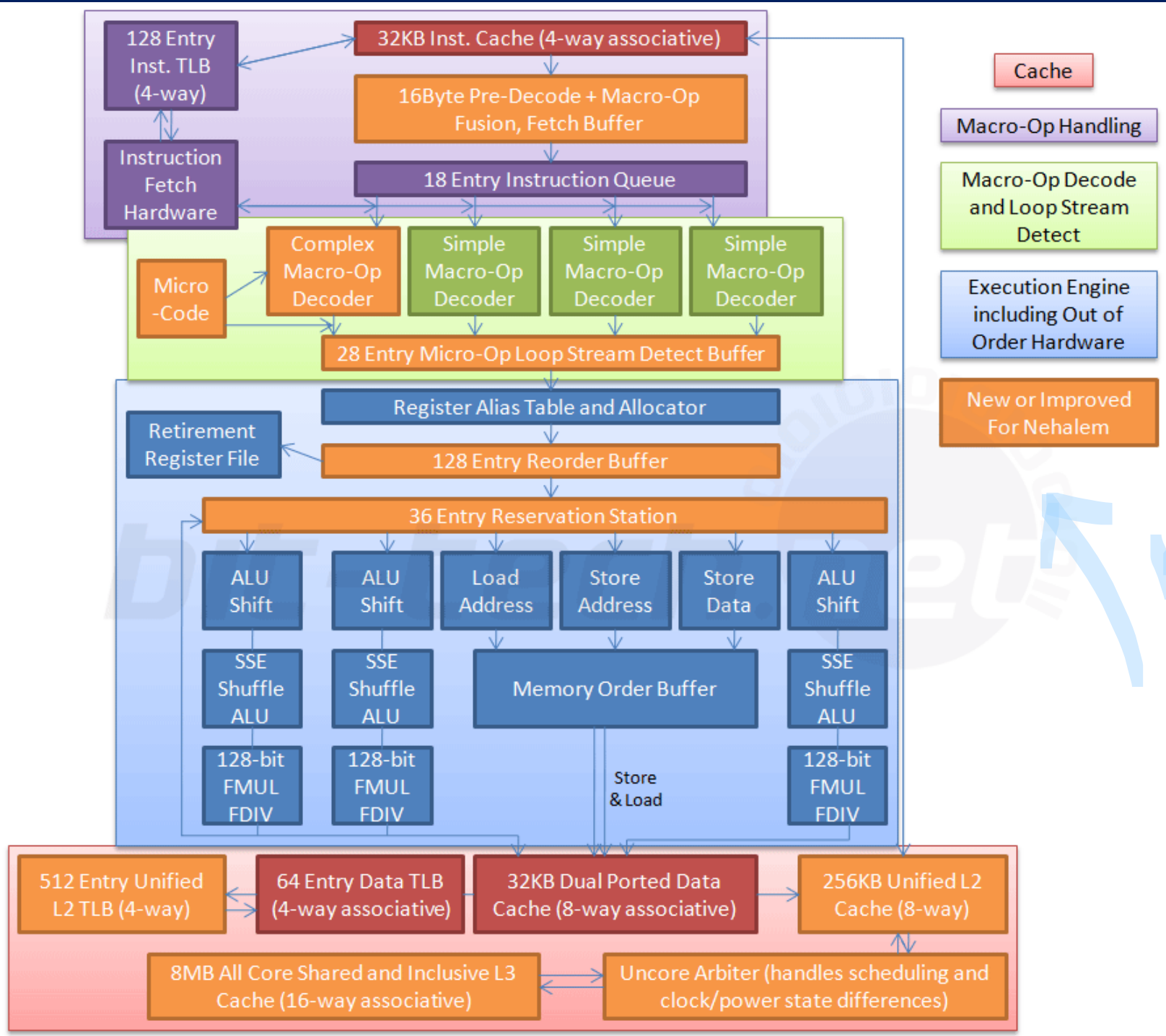


Figure 3.38 The Intel Core i7 pipeline structure shown with the memory system components. The total pipeline depth is 14 stages, with branch mispredictions typically costing 17 cycles, with the extra few cycles likely due to the time to reset the branch predictor. The six independent functional units can each begin execution of a ready micro-op in the same cycle. Up to four micro-ops can be processed in the register renaming table.

Core i7


128 Entry Inst. TLB (4-way)
32KB Inst. Cache (4-way associative)
16Byte Pre-Decode + Macro-Op Fusion, Fetch Buffer
Instruction Fetch Hardware
18 Entry Instruction Queue
Micro -Code
Complex Macro-Op Decoder
Simple Macro-Op Decoder
Simple Macro-Op Decoder
Simple Macro-Op Decoder
28 Entry Micro-Op Loop Stream Detect Buffer
Register Alias Table and Allocator
Retirement Register File
128 Entry Reorder Buffer
36 Entry Reservation Station
ALU Shift
ALU Shift
Load Address
Store Address
Store Data
ALU Shift
SSE Shuffle ALU
SSE Shuffle ALU
Memory Order Buffer
SSE Shuffle ALU
128-bit FMUL FDIV
128-bit FMUL FDIV
Store & Load
128-bit FMUL FDIV
512 Entry Unified L2 TLB (4-way)
64 Entry Data TLB (4-way associative)
32KB Dual Ported Data Cache (8-way associative)
256KB Unified L2 Cache (8-way)
8MB All Core Shared and Inclusive L3 Cache (16-way associative)
Uncore Arbiter (handles scheduling and clock/power state differences)
Cache
Macro-Op Handling
Macro-Op Decode and Loop Stream Detect
Execution Engine including Out of Order Hardware
New or Improved For Nehalem

Intel Core i7

- The Intel Core i7 pipeline structure shown with the memory system components.
- The total pipeline depth is 14 stages,
- with branch mispredictions typically costing 17 cycles, with the extra few cycles likely due to the time to reset the branch predictor.
- The six independent functional units can each begin execution of a ready micro-op in the same cycle.
- 6 Wide Superscalar
- 48 Load and 3 Store Buffers
- Up to four micro-ops can be processed in the register renaming table.

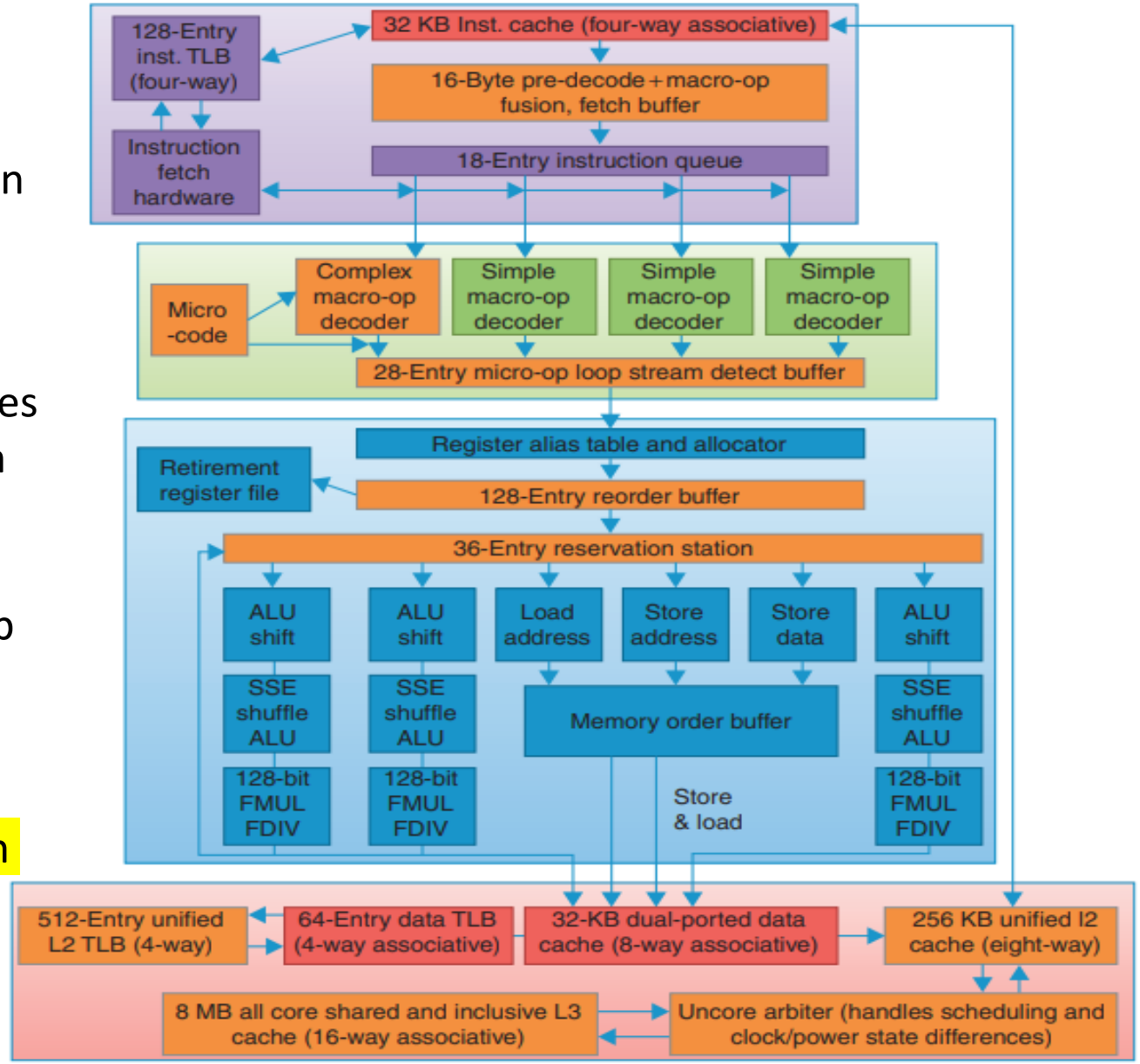
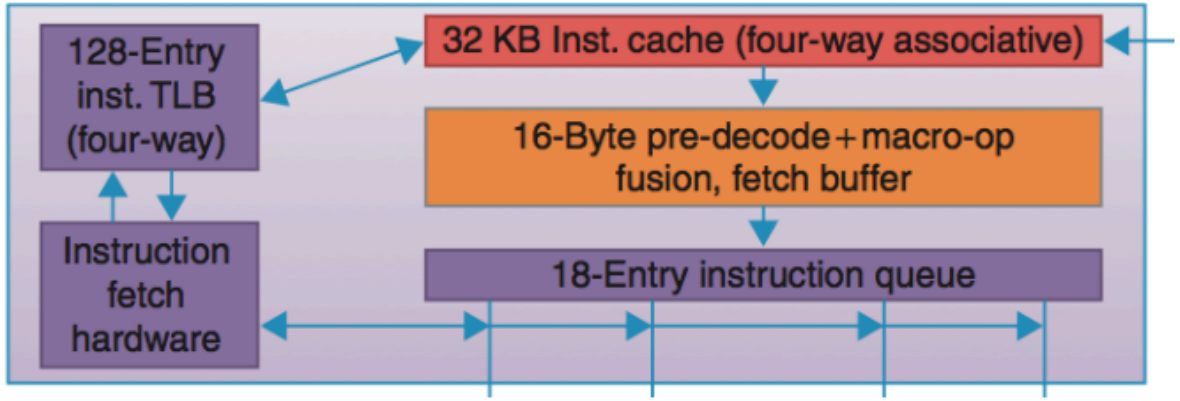


Figure 3.41 The Intel Core i7 pipeline structure shown with the memory system components. The total pipeline depth is 14 stages, with branch mispredictions costing 17 cycles. There are 48 load and 32 store buffers. The six independent functional units can each begin execution of a ready micro-op in the same cycle.

Core i7 Pipeline: IF

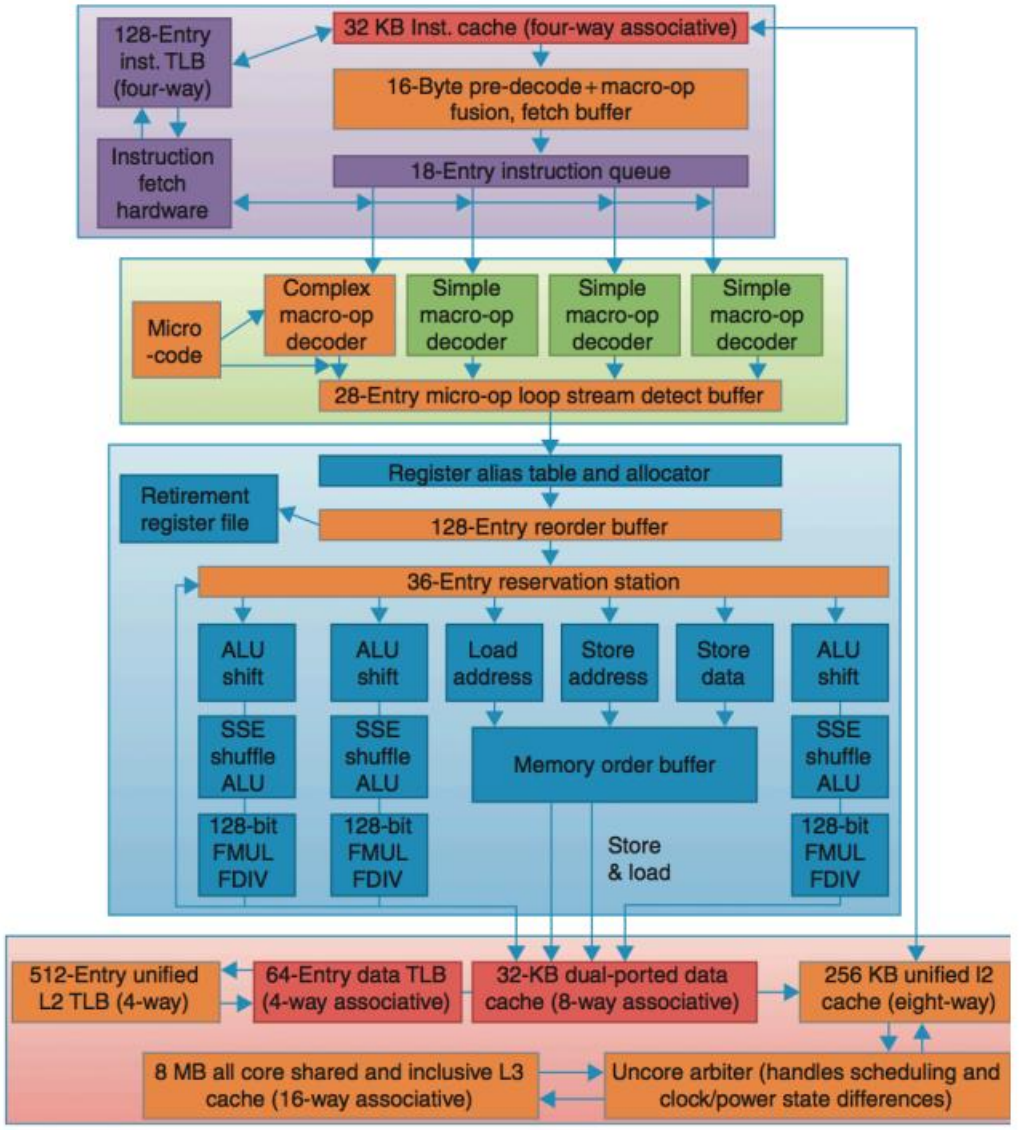


- **Instruction fetch – Fetch 16 bytes from the I cache**
  - **A multilevel branch target buffer to achieve a balance between speed and prediction accuracy.**
  - **A return address stack to speed up function return.**
  - **Mispredictions cause a penalty of about 15 cycles.**

Core i7 Architecture Features

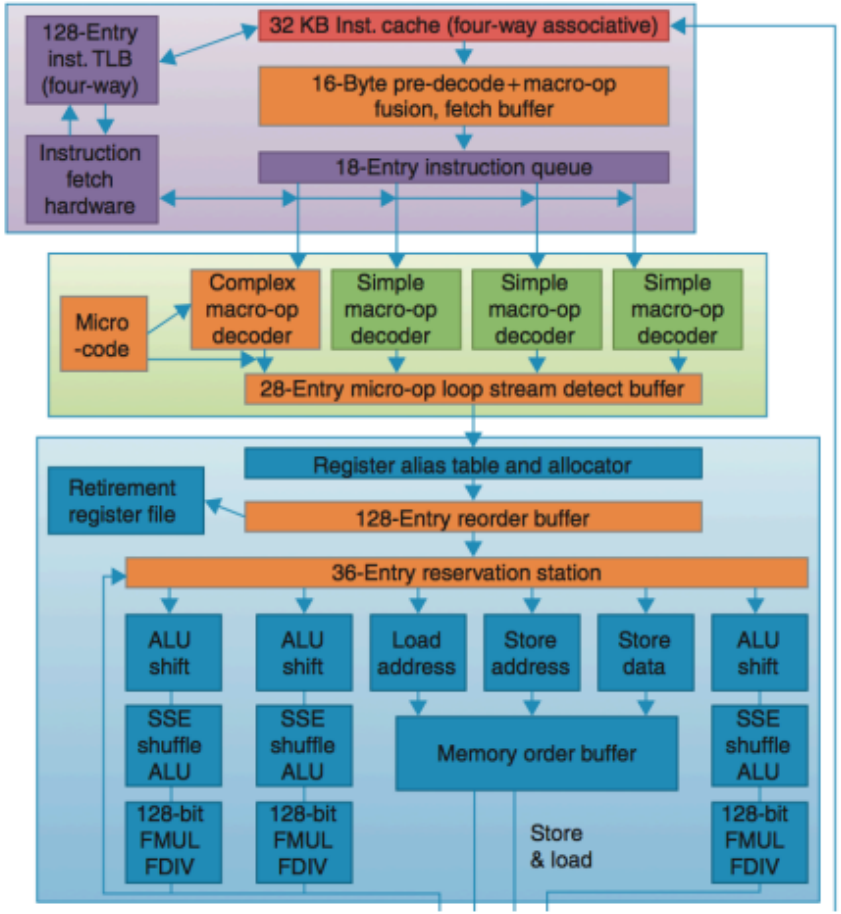
Intel Core i7

- Aggressive out-of-order speculative
- 14 stages pipeline,
- Branch mispredictions costing 17 cycles.
- 48 load and 32 store buffers.
- Six independent functional units
  - 6-wide superscalar



Core i7 Pipeline: Issue

- **Basic instruction issue**
  - Looking up the register location in the register tables
  - renaming the registers
  - allocating a reorder buffer entry
  - fetching any results from the registers or reorder buffer before send reservation stations.



- **36-entry centralized reservation station shared by six functional units**

  Up to six micro-ops may be dispatched to the functional units every clock cycle.

Dispatch / Execute Unit

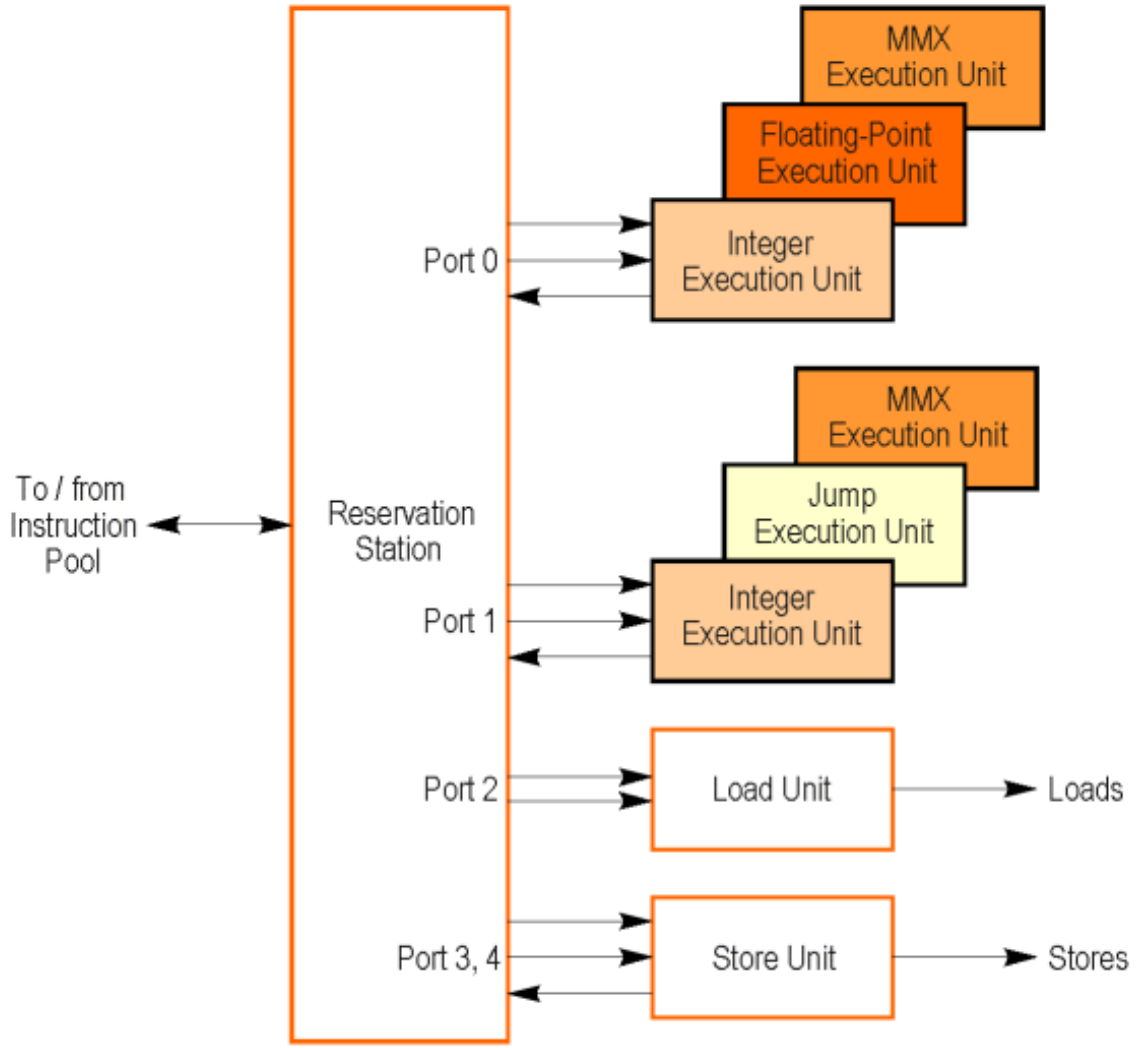
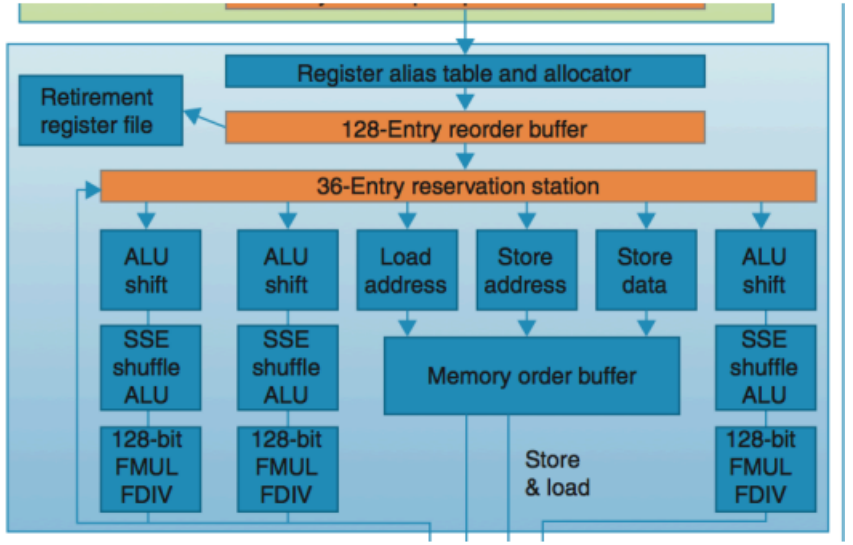


Figure 5.18. Intel Architecture processors dispatch/execute unit.

Pipeline EXE Unit

Core i7 Pipeline: EXE and Retirement



- **Micro-ops are executed by the individual function units**
  - **results are sent back to any waiting reservation station as well as to the register retirement unit, where they will update the register state. The entry corresponding to the instruction in the reorder buffer is marked as complete.**
- **Retirement**
  - **When one or more instructions at the head of the reorder buffer have been marked as complete, the pending writes in the register retirement unit are executed, and the instructions are removed from the reorder buffer.**

8 Steps in Instruction Execution (1 to 4)

Below are the eight steps an x86 instruction goes through for execution.

1. Instruction fetch—The processor uses a multilevel branch target buffer to achieve a balance between speed and prediction accuracy. There is also a return address stack to speed up function return. Mispredictions cause a penalty of about15 cycles. Using the predicted address, the instruction fetch unit fetches 16 bytesfrom the instruction cache.

2. The 16 bytes are placed in the predecode instruction buffer— The predecode stage transforms the 16 bytes into individual x86 instructions. This predecode is nontrivial since the length of an x86 instruction can be from 1 to 15 bytes and the predecoder must look through a number of bytes before it knows the instruction length. Individual x86 instructions are placed into the 18-entry instruction queue.

3. Micro-op decode—Individual x86 instructions are translated into microoperations (micro-ops). Three of the decoders handle x86 instructions that translate directly into one micro-op. For x86 instructions that have more complex semantics, there is a microcode engine that is used to produce the micro-op sequence; it can produce up to four micro-ops every cycle and continues until the necessary micro-op sequence has been generated. Th e micro-ops are placed according to the order of the x86 instructions in the 28-entry micro-op buffer.

4. The micro-op buffer performs loop stream detection—If there is a small sequence of instructions (less than 28 instructions or 256 bytes in length) that comprises a loop, the loop stream detector will find the loop and directly issue the micro-ops from the buffer, eliminating the need for the instruction fetch and instruction decode stages to be activated.

8 Steps in Instruction Execution (5 to 8)

5. Perform the basic instruction issue—Looking up the register location in the register tables, renaming the registers, allocating a reorder buffer entry, and fetching any results from the registers or reorder buff er before sending the micro-ops to the reservation stations.

6. The i7 uses a 36-entry centralized reservation station shared by six functional units. Up to six micro-ops may be dispatched to the functional units every clock cycle.

7. The individual function units execute micro-ops and then results are sent back to any waiting reservation station as well as to the register retirement unit, where they will update the register state, once it is known that the instruction is no longer speculative. The entry corresponding to the instruction in the reorder buffer is marked as complete.

8. When one or more instructions at the head of the reorder buffer have been marked as complete, the pending writes in the register retirement unit are executed, and the instructions are removed from the reorder buffer. Hardware in the second and fourth steps can combine or fuse operations together to reduce the number of operations that must be performed. Macro-op fusion in the second step takes x86 instruction combinations, such as compare followed by a branch, and fuses them into a single operation. Microfusion in the fourth step combines micro-operation pairs such as load/ALU operation and ALU operation/store and issues them to a single reservation station (where they can still issue independently), thus increasing the usage of the buffer. In a study of the Intel Core architecture, which also incorporated microfusion and macrofusion, Bird etal. [2007] discovered that microfusion had little impact on performance, while macrofusion appears to have a modest positive impact on integer performance and little impact on floating-point performance.

Pipeline Performance Core i7

1. Instruction fetch—The processor uses a sophisticated multilevel branch predictor to achieve a balance between speed and prediction accuracy. There is also a return address stack to speed up function return. Mispredictions cause a penalty of about 17 cycles. Using the predicted address, the instruction fetch unit fetches 16 bytes from the instruction cache.
2. The 16 bytes are placed in the predecode instruction buffer—In this step, a process called macro-op fusion is executed. *Macro-op fusion* takes instruction combinations such as compare followed by a branch and fuses them into a single operation, which can issue and dispatch as one instruction. Only certain special cases can be fused, since we must know that the only use of the first result is by the second instruction (i.e., compare and branch). In a study of the Intel Core architecture (which has many fewer buffers), Bird et al. (2007) discovered that macrofusion had a significant impact on the performance of integer programs resulting in an 8%–10% average increase in performance with a few programs showing negative results. There was little impact on FP programs; in fact, about half of the SPECFP benchmarks showed negative results from macro-op fusion. The predecode stage also breaks the 16 bytes into individual x86 instructions.

Buffers and Queues in Core i7

Intel Core i7: 970 (Nehalem, 2008) vs 6700 (Skylake, 2015)

Resource	i7 920 (Nehalem)	i7 6700 (Skylake)
Micro-op queue (per thread)	28	64
Reservation stations	36	97
Integer registers	NA	180
FP registers	NA	168
Outstanding load buffer	48	72
Outstanding store buffer	32	56
Reorder buffer	128	256

Figure 3.39 The buffers and queues in the first generation i7 and the latest generation i7. Nehalem used a reservation station plus reorder buffer organization. In later microarchitectures, the reservation stations serve as scheduling resources, and register renaming is used rather than the reorder buffer; the reorder buffer in the Skylake microarchitecture serves only to buffer control information. The choices of the size of various buffers and renaming registers, while appearing sometimes arbitrary, are likely based on extensive simulation.

CPI for i7 6700 and i7 920 versions

Core i7 Performance

- The integer CPI values range from 0.44 to 2.66 with a standard deviation of 0.77
- The FP CPU is from 0.62 to 1.38 with a standard deviation of 0.25.
- Cache behavior is major contribution to the stall CPI

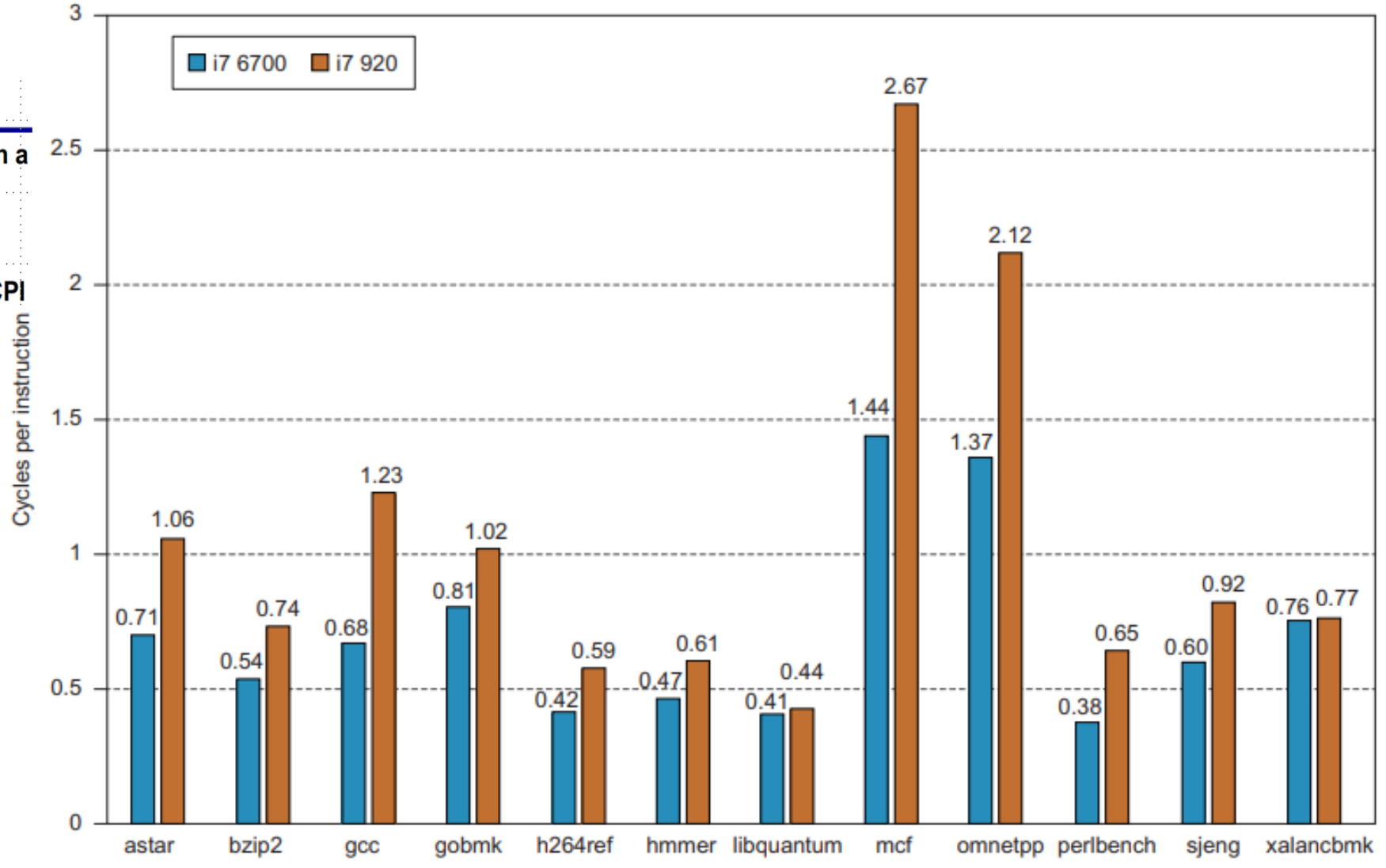


Figure 3.40 The CPI for the SPECCPUint2006 benchmarks on the i7 6700 and the i7 920. The data in this section were collected by Professor Lu Peng and PhD student Qun Liu, both of Louisiana State University.

Core i7 Performance

- The integer CPI values range from 0.44 to 2.66 with a standard deviation of 0.77
- The FP CPU is from 0.62 to 1.38 with a standard deviation of 0.25.
- Cache behavior is major contribution to the stall CPI

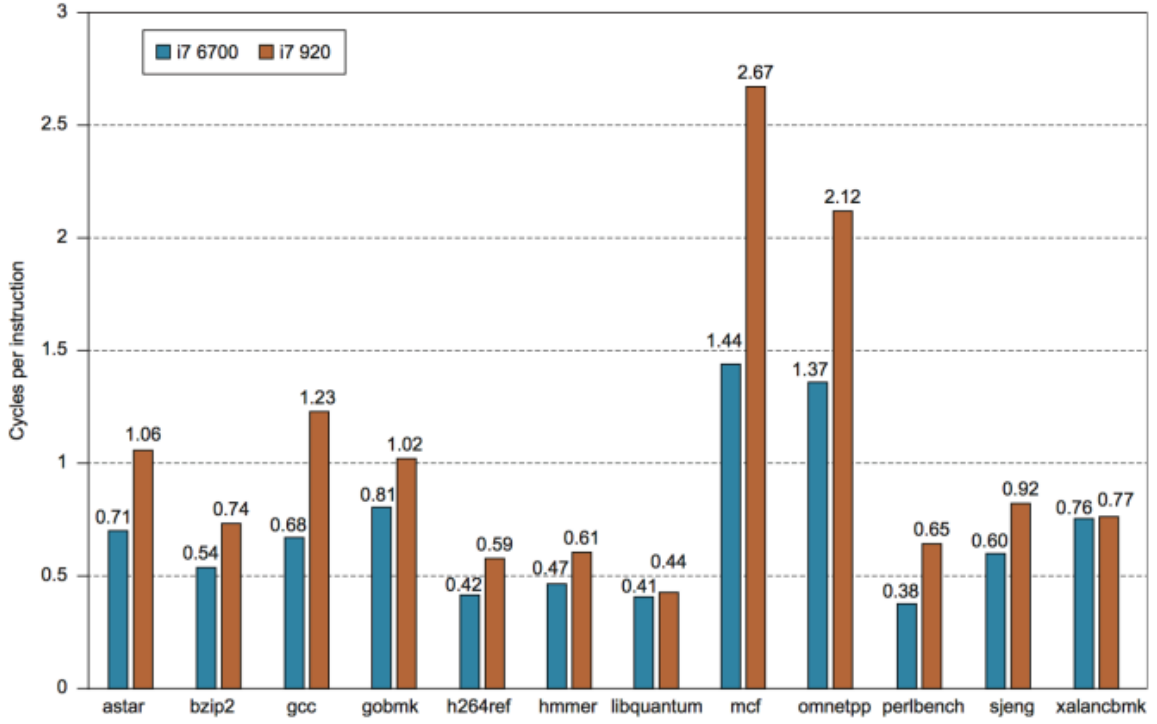


Figure 3.40 The CPI for the SPECCPUint2006 benchmarks on the i7 6700 and the i7 920. The data in this section were collected by Professor Lu Peng and PhD student Qun Liu, both of Louisiana State University.

Core i7 Predictor

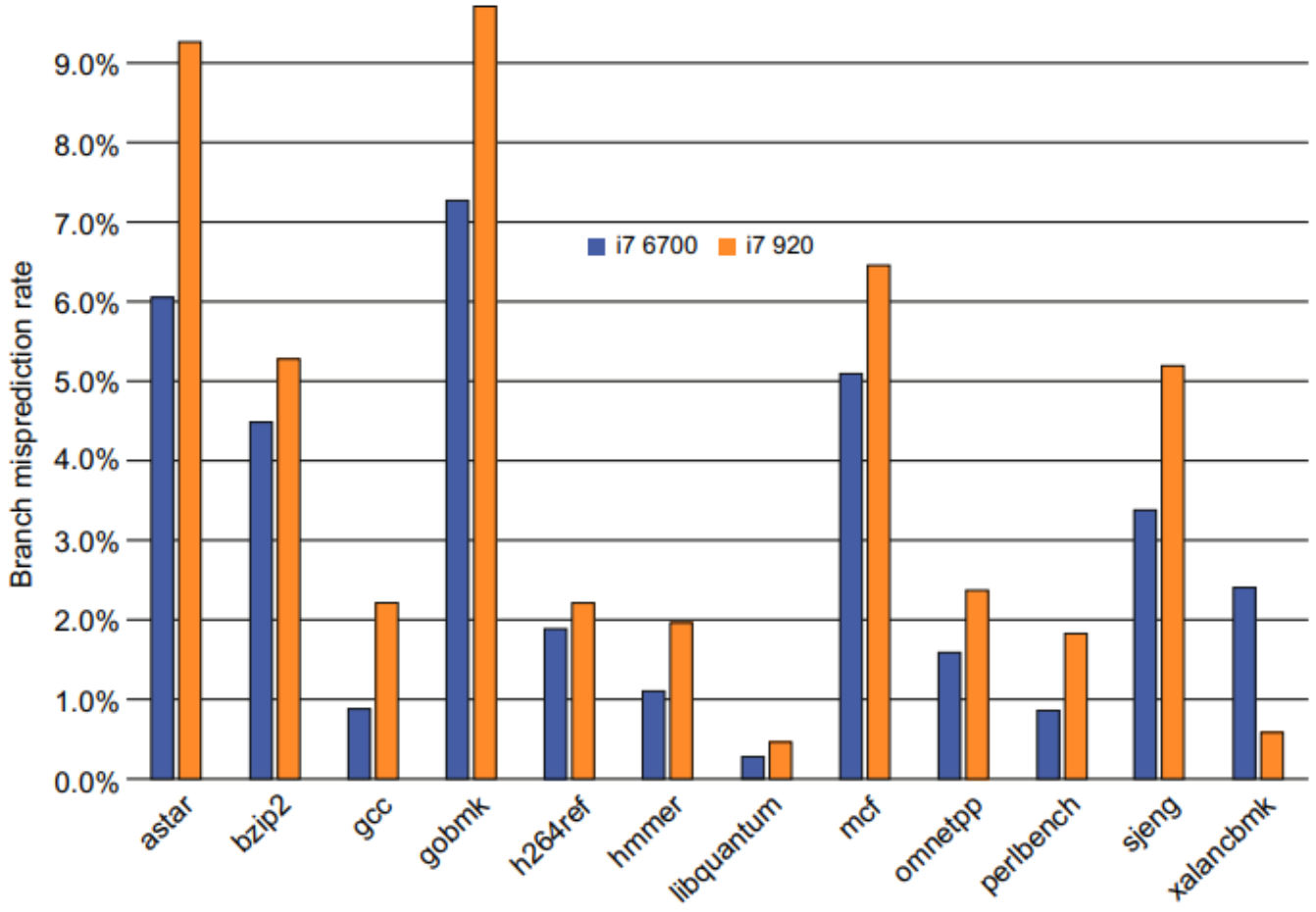


Figure 3.9 The misprediction rate for the integer SPECCPU2006 benchmarks on the Intel Core i7 920 and 6700. The misprediction rate is computed as the ratio of completed branches that are mispredicted versus all completed branches. This could understate the misprediction rate somewhat because if a branch is mispredicted and led to another mispredicted branch (which should not have been executed), it will be counted as only one misprediction. On average, the i7 920 mispredicts branches 1.3 times as often as the i7 6700.

Integer Benchmarks Comparison i7 6700 vs 920

Benchmark	CPI ratio (920/6700)	Branch mispredict ratio (920/6700)	L1 demand miss ratio (920/6700)
ASTAR	1.51	1.53	2.14
GCC	1.82	2.54	1.82
MCF	1.85	1.27	1.71
OMNETPP	1.55	1.48	1.96
PERLBENCH	1.70	2.11	1.78

Figure 3.41 An analysis of the five integer benchmarks with the largest performance gap between the i7 6700 and 920. These five benchmarks show an improvement in the branch prediction rate and a reduction in the L1 demand miss rate.

Three Different Intel Processors

Processor	Implementation technology	Clock rate	Power	SPECCInt2006 base	SPECCFP2006 baseline
Intel Pentium 4 670	90 nm	3.8 GHz	115 W	11.5	12.2
Intel Itanium 2	90 nm	1.66 GHz	104 W approx. 70 W one core	14.5	17.3
Intel i7 920	45 nm	3.3 GHz	130 W total approx. 80 W one core	35.5	38.4

Figure 3.44 Three different Intel processors vary widely. Although the Itanium processor has two cores and the i7 four, only one core is used in the benchmarks; the Power column is the thermal design power with estimates for only one core active in the multicore cases.

Compare ARM and Intel Processors

Area	Specific characteristic	Intel i7 920 Four cores, each with FP	ARM A8 One core, no FP	Intel Atom 230 One core, with FP
Physical chip properties	Clock rate	2.66 GHz	1 GHz	1.66 GHz
	Thermal design power	130 W	2 W	4 W
	Package	1366-pin BGA	522-pin BGA	437-pin BGA
Memory system	TLB	Two-level All four-way set associative 128 I/64 D 512 L2	One-level fully associative 32 I/32 D	Two-level All four-way set associative 16 I/16 D 64 L2
	Caches	Three-level 32 KiB/32 KiB 256 KiB 2–8 MiB	Two-level 16/16 or 32/32 KiB 128 KiB–1 MiB	Two-level 32/24 KiB 512 KiB
	Peak memory BW	17 GB/s	12 GB/sec	8 GB/s
Pipeline structure	Peak issue rate	4 ops/clock with fusion	2 ops/clock	2 ops/clock
	Pipe line scheduling	Speculating out of order	In-order dynamic issue	In-order dynamic issue
	Branch prediction	Two-level	Two-level 512-entry BTB 4 K global history 8-entry return stack	Two-level

Figure 3.42 An overview of the four-core Intel i7 920, an example of a typical ARM A8 processor chip (with a 256 MiB L2, 32 KiB L1s, and no floating point), and the Intel ARM 230, clearly showing the difference in design philosophy between a processor intended for the PMD (in the case of ARM) or netbook space (in the case of Atom) and a processor for use in servers and high-end desktops. Remember, the i7 includes four cores, each of which is higher in performance than the one-core A8 or Atom. All these processors are implemented in a comparable 45 nm technology.